

Senior iOS Interview Guide

Worked from a real codebase — AlarmDM 3.1

Marko Stajić

September 2026

Contents

How to use this	2
The shape of each chapter	2
What an interviewer is actually testing	2
Part I – The system	3
1. The two-minute description	3
2. One engine, many faces	5
Part II – Hard problems, worked	8
3. Identity: ids from a feed that has none	8
4. SwiftData: two stores, and what CloudKit forbids	11
5. Conflict resolution without a server	15
6. Where a listen is written down — the CarPlay bug	19
7. AVPlayer: five bugs and their shapes	23
8. Stream metadata, and giving it a lifetime	28
9. Domain rules as values	31
Part III – Four faces, one codebase	35
10. iPhone, iPad, Mac and CarPlay from one target	35
11. Localization, and forcing a default	40
12. Analytics that measure nobody	43
Part IV – Quality	46
13. Testing beyond the simulator	46
14. Reading a code review like a senior	51
15. Honest weaknesses	55
Part V – Drills	58
16. Question bank	58
17. System design: “design a podcast player”	62
18. Behavioural stories, from real commits	64
Part VI – Modern Swift and SwiftUI lab	66
19. SwiftUI: identity, ownership and updates	66
20. Concurrency: execution, cancellation and isolation	69
21. A three-to-five-minute project walkthrough	72
22. Interview cheat sheet	73
Primary references	74
Appendix A – File map	75
Numbers worth having ready	75
Appendix B – Recurring themes	77
Appendix C – Rebuilding this document	78

How to use this

This is not a list of iOS trivia. It is one real application taken apart, so that the answers you give in an interview are answers you have already lived through rather than answers you have read.

The application is **AlarmDM 3.1** — a radio and podcast client for the Serbian station *Daško i Mlada*. It streams live radio, it plays and downloads a back catalogue of several thousand episodes, it runs on iPhone, iPad, Mac Catalyst and CarPlay, it syncs what a person did with their episodes through CloudKit, and it lets them bookmark a moment while driving. About 10 900 lines of Swift, of which roughly 1 600 are tests.

That size is the point. It is small enough that one person holds all of it in their head, and large enough to have produced every category of problem a senior interview asks about: identity across devices, conflict resolution without a server, an audio pipeline with real timing bugs, a second scene in the same process with no window behind it, localization, privacy, and a code review that found things.

A note on language. The guide is in English because the interviews are. Rehearse out loud in English; the ideas are yours either way. (*Vodič je na engleskom jer su i intervjui na engleskom — ideje su svakako tvoje.*)

The shape of each chapter

Parts I–V are the walkthrough, and each of their chapters has the same four parts — you can read only the last one if you are short of time. Part VI is different in kind: a Q&A lab on modern SwiftUI and concurrency, where “current approach” means code in this app and “alternative” means a design being proposed rather than one that shipped.

1. **The problem** — what actually went wrong, in user-visible terms.
2. **The code** — the real excerpt, from the file named above it.
3. **Why it is shaped that way** — the trade-off, including the option that was rejected.
4. **In an interview** — the compressed answer, and the follow-up questions a good interviewer will ask next.

What an interviewer is actually testing

Worth keeping in mind through all of it. A senior iOS interview is rarely testing whether you know an API. It is testing four things:

- **Can you find a bug you cannot see?** Most of this codebase’s interesting history is bugs with no crash log: audio that played from the wrong second, a listen that was never written down, two devices that both thought they were right.
- **Do you know what you traded away?** Every decision here has a rejected alternative. Being able to name it is most of the difference between a mid and a senior answer.
- **Can you draw the boundary?** Which layer owns a rule, and what happens the day a second caller appears. The CarPlay chapter is entirely about this.
- **Do you know what you do not know?** Chapter 15 is a list of this codebase’s real weaknesses. Volunteering one of those, unprompted, does more for you than any correct answer.

Part I — The system

1. The two-minute description

Every interview opens with some version of “*tell me about something you have built.*” Two minutes, no whiteboard. Here is the shape that works, and then the content.

The shape: **what it is → the one hard thing → what you would do differently.** Not a feature list. A feature list invites no follow-up questions, and follow-up questions are the interview.

The content

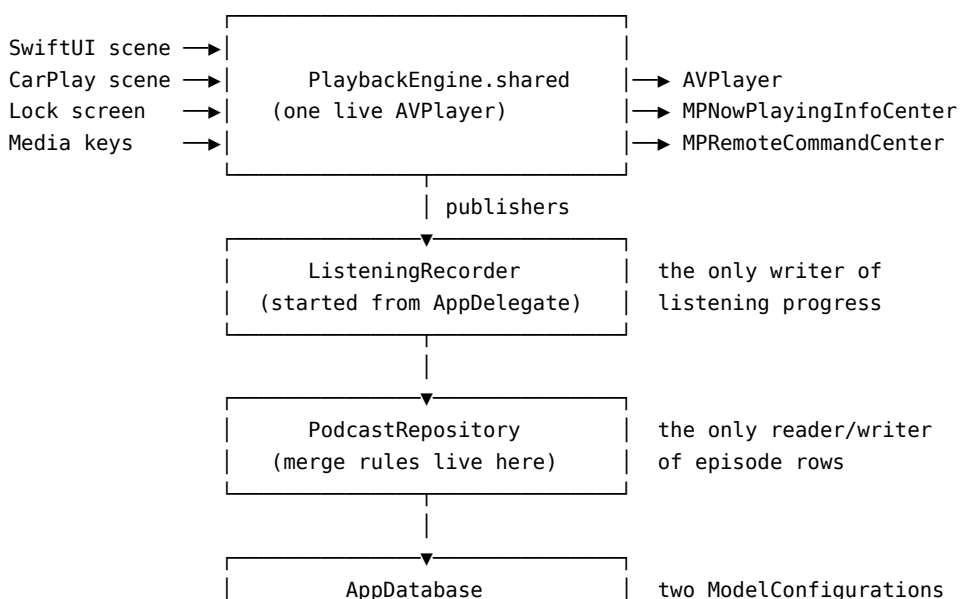
AlarmDM is a radio and podcast client for a Serbian station. Live stream, a back catalogue of a few thousand episodes, downloads for offline, and it runs on iPhone, iPad, Mac and CarPlay from one codebase. SwiftUI, with AVFoundation underneath and SwiftData plus CloudKit for state.

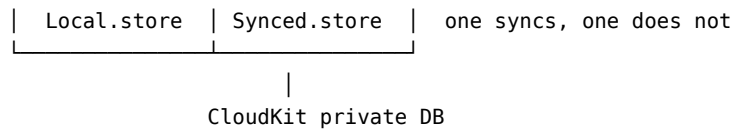
The hard part was not playback. It was that “where I got to in this episode” has to be true on four surfaces at once, with no server of my own holding the answer. The phone writes it, the car writes it, the Mac writes it, and CloudKit’s private database is the only thing between them — which has no unique constraints, no transactions across devices, and nothing you can use as event order. So the merge rules had to live in the app, and they had to be the *same* rules wherever a position is read.

The thing I would change: the store layer is main-thread-by-convention rather than by the compiler. It behaves today; nothing proves it keeps behaving, and turning on strict concurrency is a real piece of work rather than a flag.

That is roughly 150 words and it hands the interviewer three threads to pull on: CloudKit conflict resolution, the multi-surface architecture, and concurrency. All three are chapters in this guide.

The architecture, in one diagram’s worth of words





Four rules hold that picture together, and each one is a chapter:

Rule	Where it lives	Chapter
One object owns the only live AVPlayer	PlaybackEngine	2
Exactly one object writes listening progress	ListeningRecorder	6
Exactly one object applies the merge rules	PodcastRepository	5
What syncs and what does not is a schema decision	AppDatabase	4

In an interview

Say: the shape above, in 150 words, then stop talking. Silence after a crisp answer is an invitation for them to choose the direction, and any direction they choose is one you have prepared.

Do not say: “it uses MVVM.” Every candidate says it, it describes almost nothing, and it invites the one follow-up you cannot win — “*what is a view model in your app, exactly?*” — unless you can answer it with a boundary rather than a definition. In this codebase the honest answer is that `PlayerViewModel` is a façade over a shared engine, and that the interesting design work happened when parts of it were taken out and given to `ListeningRecorder`. That is chapter 6, and it is a far better story.

2. One engine, many faces

The problem

An audio app on iOS is driven from places that do not know about each other. The phone UI, the CarPlay list, the lock screen, the Mac's media keys, the car's steering-wheel buttons, and the system itself when a phone call arrives. The naive architecture — a player owned by the player screen — fails the first time one of those exists without the screen.

In this app it failed hard: **CarPlay can start the app with no phone window at all.** iOS brings up the `CPTemplateApplicationScene` and nothing else. Every object that lived on a SwiftUI view's lifetime simply was not there.

The code

`PlaybackEngine` is a singleton, and the file says why on line one:

```
// Single source of truth for audio playback.
//
// Both the SwiftUI app and the CarPlay scene drive this one object, so the
// phone and the car always show the same thing and only one AVPlayer ever
// exists. Owns the audio session, the Now Playing info and the remote
// command centre.

final class PlaybackEngine: NSObject, ObservableObject, PlaybackEngineType {
    static let shared = PlaybackEngine()
```

The view model does not own playback; it mirrors it.

```
// A thin, observable façade over PlaybackEngine.shared. It owns the UI-only
// concerns (presentation, download progress) and mirrors playback state from
// the engine, so the phone UI and the CarPlay scene can never disagree about
// what is playing.
```

And the CarPlay delegate reaches the same object, not a copy:

```
private var engine: PlaybackEngine { .shared }
```

Why it is shaped that way

The rejected alternative is instructive. The “correct” modern answer is dependency injection all the way down: no singletons, an engine created at composition root and passed through the environment. That is genuinely better for testability, and this codebase moves *towards* it — but it does not get there, for a reason worth stating plainly in an interview:

There is no single composition root. iOS builds the CarPlay scene and the SwiftUI scene independently, and a scene delegate is instantiated by the system from a class name in the `Info.plist`:

```
<key>UISceneDelegateClassName</key>
<string>$(PRODUCT_MODULE_NAME).CarPlaySceneDelegate</string>
```

You cannot hand it anything. So either the engine is global, or every scene reaches a global registry to find it — which is a singleton wearing a different hat.

What the codebase does instead is **inject at the seam that matters**. The engine is a singleton, but everything that needs to be *tested* against it depends on a protocol, not on the class:

```

/// What the player screens actually need from the engine. It exists so the
/// view model can be driven by something that is not an AVPlayer: every rule
/// about swapping episodes, keeping positions and restoring state is decided
/// in the view model, and testing it against the real engine would mean
/// testing AVFoundation as well.
protocol PlaybackEngineType: AnyObject {
    var source: PlaybackSource? { get }
    var duration: TimeInterval { get }
    var progress: Double { get }

    var sourcePublisher: AnyPublisher<PlaybackSource?, Never> { get }
    var isPlayingPublisher: AnyPublisher<Bool, Never> { get }
    var currentTimePublisher: AnyPublisher<TimeInterval, Never> { get }
    var movedByHandPublisher: AnyPublisher<TimeInterval, Never> { get }

    func play(_ source: PlaybackSource, startingAt position: TimeInterval?)
    func seek(to time: TimeInterval, completion: (() -> Void)?)
    func moveByHand(to time: TimeInterval)
    // ...
}

```

Note the comment's last line: *"The engine itself is unchanged by this — it is the only thing that implements it, and it implements it by already having these members."* That is the honest description of a seam added for testing. It costs nothing at runtime and it buys a FakePlaybackEngine (chapter 13).

The detail that makes or breaks this design

PlaybackSource is an enum with an unusual equality story, and it is the single highest-value paragraph in this chapter:

```

enum PlaybackSource: Equatable {
    case radio(url: URL)
    case podcast(Podcast)

    /// What is playing, rather than what is known about it.
    ///
    /// Two of these can hold the same audio and still be unequal: a Podcast
    /// carries a favourite flag, a downloaded file and how far it has been
    /// listened to, and every one of those changes while the episode plays.
    /// Comparing the whole value to decide whether something is already
    /// loaded answers "no" the moment any of it is written down - and the
    /// answer to that question decides between carrying on and starting the
    /// file again from the beginning.
    var contentId: String {
        switch self {
            case .radio(let url): return url.absoluteString
            case .podcast(let podcast): return podcast.id.uuidString
        }
    }

    func isSameContent(as other: PlaybackSource?) -> Bool {
        guard let other else { return false }
        return contentId == other.contentId
    }
}

```

The bug this prevents is a beautiful one to describe out loud. Podcast is a value type carrying both *what the episode is* and *what you did with it*. The moment progress is written, a refreshed Podcast is no longer == to the one the engine holds — even though it is bit-for-bit the same audio file. Any code asking *"is this already playing?"* with == gets "no", reloads the item, and the episode restarts from zero. **Identity is about the audio; equality is about the whole value; the two are not the same question.**

In an interview

The question this answers: *"When is a singleton acceptable?"*

The answer: when the thing genuinely is unique in the process and the process has more than one entry point you do not control. One `AVPlayer`, one `AVAudioSession`, one `MPNowPlayingInfoCenter` — these are not design choices, they are facts about the platform. Wrapping them in an injected object that is constructed once and then reached globally anyway is ceremony. What matters is that *consumers* depend on a protocol so they can be tested, and that the singleton owns nothing a test needs to control.

The follow-up to expect: *"How do you test it, then?"* — chapter 13.

The second follow-up: *"What about thread safety?"* — be honest, and be specific about what you have actually checked. The engine intends main-thread ownership: every KVO callback hops through `DispatchQueue.main.async` before touching state, and the seek completions do now too, which they did not until a review went looking. That covers the paths anybody has audited. It does not prove the rest, and `@Published` on a class that is only main-thread by habit is an invariant the compiler is not being asked to check. `@MainActor` on the engine would make the contract checkable; the reason it is not there yet is in chapter 15. Saying all that before they ask is worth more than any answer you could give after — and "I know which callbacks I have audited and which I have not" is a better sentence than "it is thread-safe."

Part II — Hard problems, worked

3. Identity: ids from a feed that has none

The problem

The backend is a Firebase Cloud Function that derives episodes from the station's RSS feed. The payload may carry an id and may not — the decode reads `response.id` when it is there and falls back to the media URL — so “there is a stable key” is a contract with the feed to be checked, not a property to assume. The first version of the model assumed the opposite and did the obvious thing:

```
var id = UUID()           // a fresh one, on every decode
```

The consequences were invisible and total. Every refresh inserted the same episode again rather than updating it. Anything written against one of those ids — a favourite, a listening position, a downloaded file reference — could never be found again, because the id it was written under no longer existed anywhere. It was not a crash, it was not slow, and nothing in the UI said so. Favourites simply did not stick.

The code

AppConstants.swift:

```
extension UUID {
    /// A UUID derived deterministically from a stable string - the episode's
    /// media URL.
    ///
    /// The API has no UUIDs, and `Podcast` previously generated a fresh one on
    /// every decode. Since that UUID was the episode's primary key, each
    /// refresh inserted the same episode again instead of updating it - and
    /// nothing written against one of those ids could ever be found again.
    /// Deriving the id from the feed keeps one row per episode across
    /// refreshes, across launches, and across devices.
    static func stable(from string: String) -> UUID {
        var bytes = Array(Insecure.MD5.hash(data: Data(string.utf8)))
        bytes[6] = (bytes[6] & 0x0F) | 0x50 // version 5-ish: derived, not random
        bytes[8] = (bytes[8] & 0x3F) | 0x80 // RFC 4122 variant
        return UUID(uuid: (bytes[0], bytes[1], bytes[2], bytes[3],
            bytes[4], bytes[5], bytes[6], bytes[7],
            bytes[8], bytes[9], bytes[10], bytes[11],
            bytes[12], bytes[13], bytes[14], bytes[15]))
    }
}
```

And at the decode site:

```
init(from response: PodcastResponse) {
    // Identity comes from the media URL, never from a fresh UUID - see UUID.stable.
    let identitySource = response.id.isEmpty ? response.podcastUrl : response.id
    self.id = .stable(from: identitySource)
```

Why it is shaped that way

Three things are worth being precise about, because an interviewer who knows this area will probe all three.

1. Why a derived UUID rather than just using the URL as the key? Because the domain is typed on `UUID` throughout — `Podcast.id`, the `podcastId` foreign key every synced row carries, CarPlay's list identity. Keeping the type and deriving the value is much less invasive than changing the key type everywhere. Be careful not to overclaim here: the CloudKit *record* name is managed by `NSPersistentCloudKitContainer` and is not our UUID. Ours is a domain identifier that happens to be stable across devices, which is all the design needs.

2. Why is this not a security problem? `Insecure.MD5` is named that for good reason, and the name is doing its job here: it warns you at every call site. But this is not authentication and not integrity — it is *bucketing a string into a 128-bit space*. Collision resistance against an adversary is irrelevant; nobody gains anything by making two episode URLs collide. What matters is determinism and a low accidental collision rate, and MD5 gives both. **Say this out loud in an interview**, because the interviewer's first reaction to MD5 is going to be a raised eyebrow, and "I know, and here is why it is the right tool for identity rather than security" is a much stronger position than switching to SHA-256 and not being able to say why.

(The honest caveat: SHA-256 truncated to 16 bytes would cost nothing and remove the conversation entirely. It is on the list in chapter 15.)

3. What do those two bit-twiddles do? They set the version and variant fields, per RFC 4122: byte 6's high nibble and byte 8's top bits. The result is a well-formed UUID rather than sixteen loose bytes, so anything that parses or validates one accepts it.

Two things not to claim. It is **not** a standards-compliant v5 — a real v5 is SHA-1 over a namespace plus a name, and the code's own comment says "version 5-ish" for exactly that reason. And masking six bits leaves **122 variable bits**, not 128; that is still far more than a few thousand episodes need, but if you say 128 to someone who knows the format, you have just told them you do not.

The payoff, which is the actual interview answer

The derivation has a property that is worth more than the bug it fixed: **two devices independently arrive at the same id for the same episode**, with no coordination and no server round-trip. That is what makes the whole sync design in chapter 5 possible. `PodcastEntity` says so:

```
// Deliberately not @Attribute(unique). CloudKit refuses unique constraints,  
// and this database is meant to sync between a person's devices later.  
// Identity is safe without it: Podcast.id is derived from the media URL  
// (UUID.stable), so two devices independently arrive at the same id for the  
// same episode, and the repository upserts by fetching first rather than  
// trusting the store to reject a duplicate.
```

In an interview

The question this answers: "How do you handle identity for data from an API that does not provide ids?" — and, more often, "tell me about a bug that was hard to find."

The compressed answer: "Derive the key deterministically from whatever the payload has that is stable — for us the media URL — rather than generating one. The bug before we did that was silent: every refresh created a duplicate row, so everything a user wrote was orphaned the moment the feed reloaded. No crash, no error, favourites just did not stick. The fix also bought us something we needed later: two devices reach the same id for the same episode without talking to each other, which is what lets CloudKit sync work without unique constraints."

Follow-ups to prepare:

- "What if the media URL changes?" Then it is a new episode as far as the app is concerned, and the user's state on it is orphaned. The mitigation is that the backend controls the URLs and they are content-addressed by date; the real answer is that this is a known, accepted failure mode, and the alternative — a server-assigned id — would mean running a server that remembers, which this project deliberately does not.
- "Collisions?" 122 variable bits over a few thousand episodes. The birthday bound is not the risk here; a feed that reuses a URL for different audio is, and so is a backend that starts sending ids later — that

- would silently re-key every episode unless the fallback order is preserved.
- “Why not *hashValue*?” Swift’s `Hashable` is explicitly seeded per-process and is not stable across launches. This is a genuinely common mistake and a good thing to know cold.

4. SwiftData: two stores, and what CloudKit forbids

The problem

Not everything a person's device holds is worth putting in their iCloud account. This app holds four kinds of row:

Row	Reproducible from	Should sync?
PodcastEntity — the feed cache	the API	no
DownloadEntity — which file is on disk	the network	no
EpisodeStateEntity — favourites, positions	nothing	yes
BookmarkEntity — a moment someone marked	nothing	yes

There are a few thousand episodes. Syncing them would spend a person's iCloud storage on something they can have for free, on every device, and would sync a local file path that is meaningless on the other machine.

The code

AppDatabase splits the schema and opens **two configurations in one container**:

```

/// What a person made: bookmarks, favourites, how far they got. Small,
/// irreplaceable, and worth carrying between their devices.
static let syncedModels: [any PersistentModel.Type] =
    [BookmarkEntity.self, EpisodeStateEntity.self]

/// What this device happens to hold: the feed cache and the downloaded
/// files. Both are reproducible - one from the API, the other from the
/// network - and a file path is a fact about one machine anyway.
static let localModels: [any PersistentModel.Type] =
    [PodcastEntity.self, DownloadEntity.self]

/// Two stores in one container: a context reaches both, and which one a
/// row lands in is decided by its type.
private static func configurations(inMemory: Bool, syncing: Bool,
                                   directory: URL? = nil) -> [ModelConfiguration] {
    let local = ModelConfiguration(
        "Local",
        schema: Schema(localModels),
        isStoredInMemoryOnly: inMemory,
        cloudKitDatabase: .none
    )

    let synced = ModelConfiguration(
        "Synced",
        schema: Schema(syncedModels),
        isStoredInMemoryOnly: inMemory,
        cloudKitDatabase: syncing ? .private(cloudContainer) : .none
    )

    return [local, synced]
}

```

The three CloudKit rules that shape the models

This is the part interviewers who have shipped a synced app will test, because it is where everyone gets burned. `NSPersistentCloudKitContainer` — which is what SwiftData's `cloudKitDatabase` sits on — refuses a schema that breaks any of these:

1. No unique constraints. There is no `@Attribute(.unique)` anywhere in the synced models. CloudKit has no mechanism to enforce uniqueness across devices, so the store will not open with one. Consequence:

duplicates are possible and the app must merge them. That is chapter 5.

2. Every attribute needs a default, every relationship must be optional. CloudKit records arrive field-by-field; a required value with no default cannot be materialised. EpisodeStateEntity shows the discipline:

```
@Model
final class EpisodeStateEntity {
    var podcastId: UUID = UUID()
    var isFavorite: Bool = false
    var playedPosition: Double = 0
    var playedAt: Date?
    var isPlayed: Bool = false
    var episodeTitle: String = ""
    var show: String?
```

Note `podcastId: UUID = UUID()` — a default that is *never meaningful*, present purely to satisfy the schema rule, with the real value always passed in `init`. PodcastEntity has the same pattern and a comment that names the cost: “*Cheap now, expensive to retrofit later.*”

3. No relationships across configurations. This one is easy to miss and it directly shapes the data model. EpisodeStateEntity cannot have a @Relationship to PodcastEntity, because they live in different stores. So the link is a plain foreign key:

```
/// Keyed by `podcastId` rather than by a relationship, because a relationship
/// cannot cross two stores and the episodes live in the local one.
```

Which in turn is why EpisodeStateEntity **duplicates** the episode title and show:

```
/// Enough to show a row on a device that has never cached this episode.
/// Copied, not looked up, for the same reason a bookmark copies its title.
var episodeTitle: String = ""
var show: String?
```

Denormalisation as a deliberate consequence of a platform constraint, not as an optimisation. That is a good sentence to have ready.

Failing softly

A schema CloudKit refuses throws at container creation. Losing the entire local database over that would be absurd, so the failure is staged:

```
do {
    container = try ModelContainer(for: schema, configurations: /* syncing */)
} catch {
    // Most often this is iCloud refusing the schema - a model that
    // breaks one of CloudKit's rules, or an entitlement missing on a
    // build. Losing the whole database over that would be absurd when
    // the same store opens perfectly well unsynced, so try again
    // without it before giving up.
    AppLog.write(.sync, "SwiftData store unavailable, retrying without iCloud: \(error.localizedDescription)")
    do {
        container = try ModelContainer(for: schema, configurations: /* not syncing */)
    } catch {
        isEphemeral = true
        // If even an in-memory container cannot be built, the schema
        // itself is wrong - a programmer error, not a runtime
        // condition, and there is nothing sensible left to fall back to.
        container = try! ModelContainer(for: schema, configurations: /* in memory */)
    }
}
```

Three tiers: **synced** → **local** → **memory**, and a `try!` at the bottom. Being able to defend a `try!` is itself a small senior signal, and the defence has to be the honest one: it is a deliberate fail-fast when there is no usable

persistence left at all. A malformed schema is the likeliest cause and not the only one, so calling it *proof* of a programmer error overstates it. The rule worth stating is that a forced operation is defensible when there is no recovery left to attempt, and indefensible as a shortcut past an error that depends on the environment.

There is a real hole here, and it is worth volunteering: `isEphemeral` is recorded, but **nothing on screen tells the user that syncing is off**. Silent degradation is the one thing this staging does badly. Chapter 15.

The context decision

```
/// Deliberately not `container.mainContext`, which is @MainActor and would
/// drag the annotation through the repository and everything that calls it.
/// A context of our own is nonisolated, and this one is only ever touched
/// from the main thread anyway - every write reaches it from a network
/// completion that already hopped there.
///
/// The trade is autosave: a hand-made context does not save on its own, so
/// every write ends in an explicit `save()``.
private(set) var context: ModelContext
```

```
context = ModelContext(container)
context.autosaveEnabled = false
```

Be ready to be pushed on this, because it is the codebase's most arguable decision. The honest framing:

It buys explicit transactions — every write ends in a `commit` that saves or rolls back, and nothing half-written is ever left in the context to be carried along by the next successful save. It costs the compiler's help. `mainContext` would have given us `@MainActor` checking for free; instead the invariant is documented and held by convention. Under Swift 6 that convention has to become an annotation.

And the commit that pairs with it:

```
/// A hand-made ModelContext does not autosave, so every change ends here.
/// On failure the edits are rolled back rather than left sitting in the
/// context, where the next successful save would carry them along.
private func commit(_ what: String) {
    do {
        try context.save()
    } catch {
        AppLog.write(.store, "Error \ \(what): \ \(error.localizedDescription)")
        context.rollback()
    }
}
```

Hearing about a change you did not make

When CloudKit imports, the rows change underneath screens that are holding snapshots taken when they appeared. Core Data has a notification for exactly this, and `SwiftData` is Core Data underneath:

```
var didChangeRemotely: AnyPublisher<Void, Never> {
    storeChanged
        .debounce(for: .milliseconds(400), scheduler: DispatchQueue.main)
        .eraseToAnyPublisher()
}
```

```
NotificationCenter.default.addObserver(
    forName: .NSPersistentStoreRemoteChange, object: nil, queue: .main
) { [weak self] _ in
    guard let self else { return }
    self.adoptStoreChanges()
    self.storeChanged.send()
}
```

The debounce is not tidiness. The comment records the measurement: *“bringing up iCloud posts this dozens of times in a couple of seconds — forty of them in five, on a fresh install — and every one would send every open screen back to the store for a list it already has. What the screens need to know is that something arrived, not how many times it arrived.”*

Two details in that block that interviewers like:

- `[weak self]` on a block-based observer is necessary but **not sufficient** — block observers are not removed when their object goes away. The tokens are kept and released:

```
/// Block observers are not removed by their object going away, so they
/// are kept and handed back. One database lives as long as the app, but
/// a test makes several and each one used to leave its blocks behind.
private var observers: [NSObjectProtocol] = []

deinit { observers.forEach(NotificationCenter.default.removeObserver) }
```

Note why it was found: tests. One database lives forever in the app, so the leak was invisible until a test suite made several.

- The app does not depend on the notification arriving. Every screen also re-reads when it appears or is pulled down, and the player re-reads on the press of play. Chapter 6’s `refreshFromStoreIfIdle` is that belt-and-braces.

In an interview

Question: *“How would you sync user data between a user’s devices without a backend?”*

Answer: “CloudKit’s private database through `NSPersistentCloudKitContainer` — SwiftData’s `cloudKitDatabase: .private`. The three things that drive the design: no unique constraints, so you must merge duplicates yourself; every attribute needs a default and every relationship must be optional; and relationships cannot cross store configurations. That last one is why we split the schema — the feed cache and the download records stay local and only the user’s own state syncs — and why the synced rows carry a foreign key plus a denormalised copy of the episode title, so a device that has never cached an episode can still draw a row for it.”

Follow-up: *“What happens when the schema is rejected?”* → the three-tier fallback, and the honest admission that the user is never told.

Follow-up: *“How do you know sync is working at all?”* → this is a great one to have an answer for, because the usual answer is “you don’t.” This codebase instruments it in `DEBUG` with `NSPersistentCloudKitContainer.eventChangedNotification`, logging every setup/import/export with its error, plus the iCloud user record id — because *“two devices exporting happily and neither ever seeing the other is exactly what two different accounts look like.”*

5. Conflict resolution without a server

The problem

Two devices listen to the same episode before either has heard from the other. Neither can create a row “if it does not exist” atomically, because CloudKit has no unique constraint (chapter 4). So both create one, both rows sync, and now **every device holds two rows for one episode**.

What happens next is what “sync is broken” actually looks like to a user: whichever row the store happens to return first is the one the device reads, forever, with the other device’s newer row sitting right beside it.

The code

Two rules, and the whole chapter is these six lines:

```
/// What several rows for one episode add up to: the later listen's
/// position, and flags from all of them.
static func effective(_ rows: [EpisodeStateEntity]) -> EpisodeState? {
    let sorted = rows.sorted { isNewer($0, than: $1) }
    guard let newest = sorted.first else { return nil }

    return EpisodeState(
        playedPosition: newest.playedPosition,
        playedAt: newest.playedAt,
        isPlayed: sorted.contains { $0.isPlayed },
        isFavorite: sorted.contains { $0.isFavorite }
    )
}
```

A position is a moment, and the later moment is the true one. A flag is a decision, and a decision made on either device stands.

Last-write-wins for the scalar; union for the booleans. The tiebreaker is explicit about what it is comparing:

```
/// Which of two rows for the same episode describes the later listen.
/// A row that has never been played has no date at all, and loses to one
/// that has.
static func isNewer(_ lhs: EpisodeStateEntity, than rhs: EpisodeStateEntity) -> Bool {
    (lhs.playedAt ?? .distantPast) > (rhs.playedAt ?? .distantPast)
}
```

And there is a sharp comment on playedAt itself:

```
/// When that position was written down, and the tiebreaker when two
/// devices disagree: CloudKit keeps whichever write landed last, which is
/// not the same as the listen that happened last.
var playedAt: Date?
```

That distinction — **arrival order is not event order** — is the reason the timestamp is in the model at all, and it is exactly the kind of sentence that marks someone who has actually shipped a synced app.

The bug that made this one rule instead of two

The rule used to exist twice, subtly differently. A list read went one way and a single episode read went another:

```
/// Both halves matter, and they used to disagree. A list took the newest
/// row whole, an episode opened on its own OR-ed the flags, so an episode
/// favoured on one device and listened to later on another was missing
/// from the favourites list until the player had been opened on it. One
/// rule now, read from both.
```

The user-visible symptom: *your favourite disappears from the favourites list until you open the episode, and then it comes back*. Now both paths call `EpisodeStateEntity.effective`:

```
// the single-episode read
func podcast(with id: UUID) -> Podcast? {
    guard let entity = entity(with: id) else { return nil }
    let rows = fetchStates(matching: #Predicate { $0.podcastId == id })
    return Podcast(from: entity,
        state: EpisodeStateEntity.effective(rows),
        download: download(for: id))
}
```

```
// the list read - one query for the whole page
let states = Dictionary(grouping: fetchStates(matching: #Predicate { ids.contains($0.podcastId) }),
    by: { $0.podcastId })
    .compactMapValues { EpisodeStateEntity.effective($0) }
```

Reading is not writing

The first fix folded duplicates away *on read* and deleted the losers. A code review caught it, and the reasoning is worth memorising because it generalises far beyond `SwiftData`:

```
/// Reading only. Duplicate rows are folded into one answer here and left
/// where they are; deleting them belongs to the next write - see merged.
```

A delete during a read is a store mutation on a code path that runs while a list is being drawn — it invalidates fetched objects, it can post a change notification that sends every screen back to the store, and it makes an innocuous-looking `podcast(with:)` a mutating operation that cannot be called from anywhere that holds results. The test states it as a requirement:

```
/// Reading is not writing: opening an episode used to fold its duplicates
/// away and delete them, which is a store change in the middle of a read.
func testReadingAnEpisodeLeavesTheRowsAlone() {
    insertRow(position: 600, at: earlier)
    insertRow(position: 1_800, at: later)

    _ = repository.podcast(with: episode.id)

    XCTAssertEqual(stateRows().count, 2)
}
```

Cleanup happens on the next write instead, which is a moment that is already a transaction:

```

/// Folds duplicate rows for one episode into the newest of them and
/// deletes the rest.
@discardableResult
private func merged(_ rows: [EpisodeStateEntity]) -> EpisodeStateEntity? {
    let sorted = rows.sorted { EpisodeStateEntity.isNewer($0, than: $1) }
    guard let winner = sorted.first else { return nil }
    guard sorted.count > 1 else { return winner }

    for duplicate in sorted.dropFirst() {
        winner.isFavorite = winner.isFavorite || duplicate.isFavorite
        winner.isPlayed = winner.isPlayed || duplicate.isPlayed
        if winner.episodeTitle.isEmpty { winner.episodeTitle = duplicate.episodeTitle }
        if winner.show == nil { winner.show = duplicate.show }
        context.delete(duplicate)
    }

    commit("merging duplicate episode state")
    return winner
}

```

The flag that cannot travel — and being honest about it

OR-ing booleans is a CRDT-shaped choice: it is commutative, associative and idempotent, so it converges no matter what order the rows arrive in. That is exactly why it is right for a lossy, unordered channel.

It also means **un-favouriting cannot sync**. If a device clears the flag, any other device still holding true will re-assert it on the next merge. The codebase names this rather than hiding it:

```

/// A position is a moment and the later moment is the true one. A flag is
/// a decision, and a decision made on either device stands - which is
/// also why unfavouriting cannot yet travel: see the note on that in
/// PodcastRepository.merged.

```

This is the single best thing in the chapter to volunteer in an interview. The proper fix is to stop storing a boolean and store a *decision with a timestamp* — `isFavorite: Bool` plus `favoritedAt: Date?`, merged by the same last-write-wins rule as the position — or a tombstone. Knowing the fix and being able to say why it was not done yet (“it is a schema migration on a synced store, which is the one migration you cannot take back, and it was not worth it before 3.1 shipped”) is a complete senior answer.

`isPlayed` is different, and deliberately so: it is *sticky by intent*, not by accident.

```

// Sticky. Starting an episode again does not make it unfinished, and
// an episode left at five minutes on a second device should not undo
// the fact that it was heard through on the first.
state.isPlayed = state.isPlayed || hasFinished

```

The tests

`SyncStateTests.swift` opens by stating what it does *not* test, which is a habit worth stealing:

```

// The half of syncing this app owns: what happens to the rows once iCloud
// has put them in the store. Which of two listens wins, which flags survive,
// and whether the next decision reads what arrived or what was already held.
//
// iCloud itself is not here. Whether a row leaves one device and reaches
// another is Apple's machinery, over the network, on its own schedule; a test
// of that needs two signed-in devices and minutes of waiting, and is a
// different kind of test from these.

```

The table of cases reads like a specification:

Test	What it pins down
testTheLaterListenIsWhatIsRead	LWW on position, whichever row comes back first
testARowWithoutADateLosesToOneWithADate	undated rows lose regardless of position
testAListAndAnEpisodeAgreeOnAFavourite	one rule, two read paths
testReadingAnEpisodeLeavesTheRowsAlone	reads do not mutate
testTheNextWriteLeavesOneRow	cleanup happens, just later
testHeardOnEitherDeviceStaysHeard	union on flags
testFinishingIsSticky	the same rule on one device
testCarryOnSkipsWhatWasFinished	“continue listening” ignores finished episodes

And “another device” is simulated by writing **straight into the store, bypassing the repository** — which is precisely what an import does:

```
/// A row as another device would have left it: written straight into the
/// store, not through the repository.
private func insertRow(position: Double, at date: Date?,
    favourite: Bool = false, played: Bool = false) {
    let row = EpisodeStateEntity(podcastId: episode.id, title: episode.title)
    row.playedPosition = position
    row.playedAt = date
    row.isFavorite = favourite
    row.isPlayed = played
    database.context.insert(row)
    XCTAssertNoThrow(try database.context.save())
}
```

In an interview

Question: “Two devices edit the same record offline. What happens?”

Answer, in four sentences: “It depends on what the field *means*. A position is a measurement of a moment, so the later moment wins — last-write-wins, with a timestamp we write ourselves, because CloudKit’s ordering is arrival order and that is not event order. A flag is a decision, so decisions from both devices are unioned, which converges regardless of delivery order. The price of the union is that clearing a flag cannot propagate, and the fix for that is to store the decision’s timestamp too, or a tombstone — we know that, we have not paid for the migration yet.”

Follow-up to expect: “Is that a CRDT?” — the boolean union has the shape of one: commutative, associative, idempotent, so it converges whatever order the rows arrive in. Say it that way only if you are comfortable, because the next question is “so what is the convergence proof?”, and there are two honest answers.

The union is trivially convergent. The position is not, quite, and this is worth volunteering because it is a real hole:

```
static func isNewer(_ lhs: EpisodeStateEntity, than rhs: EpisodeStateEntity) -> Bool {
    (lhs.playedAt ?? .distantPast) > (rhs.playedAt ?? .distantPast)
}
```

An LWW register converges only on a **total** order, and this comparator is strict: two rows with the same playedAt are “not newer” in both directions, so which one wins is whatever order the fetch returned. In practice two devices do not write the same millisecond, and two rows that *did* would almost certainly hold the same position — but “almost certainly” is not a proof, and the fix is cheap: break the tie on a stable writer id. Clock skew is the second assumption underneath it, and chapter 15 names it.

6. Where a listen is written down – the CarPlay bug

The problem

This is the most valuable story in the codebase and it is worth telling slowly.

The symptom, in the user's words: *"I resumed the episode in the car at 57:45. I drove for fifteen minutes. Next time I got in, it was back at 57:45."* Every time. Only in the car.

Nothing crashed. Nothing was slow. The audio itself was perfect – it played from 57:45, it advanced normally, it paused correctly when the cable was pulled. **The fifteen minutes of listening were simply never written down.**

Why

Recording progress was the player screen's job. It lived in `PlayerViewModel`, which exists because a SwiftUI view created it.

When CarPlay starts the app, iOS brings up the `CPTemplateApplicationScene` and **nothing else**. No `WindowGroup`, no `RootView`, no `PlayerViewModel`. The engine was there – the CarPlay delegate reaches `PlaybackEngine.shared` directly – so audio worked perfectly. The object that would have written the position down had never been constructed.

Pulling the cable paused the audio correctly, and nobody wrote the pause down. So the next start read the last position anyone *had* written, which was whatever the phone had last been paused at by hand.

The fix: move the job to something that always exists

```
// The one place that writes down how far a listen got. It watches the
// engine, not a screen, so a listen is written down wherever it happened.

/// This used to be the player screen's job, and the player screen does not
/// always exist. When the car starts the app, iOS brings up the CarPlay scene
/// and nothing else - no phone window, no view model - and every listen in
/// the car went unrecorded. Pulling the cable paused the audio correctly and
/// nobody wrote the pause down, so the next start went back to wherever the
/// phone had last been paused by hand. The engine exists whichever way the
/// app was started, and so does this.
final class ListeningRecorder {
    static let shared = ListeningRecorder()
```

Started from the one place that runs however the app was launched:

```
func application(_ application: UIApplication,
    didFinishLaunchingWithOptions ...) -> Bool {
    // Here and not in a view: when the car starts the app there is no
    // phone window at all, and the listen still has to be written down.
    AppLanguage.applyDefaultOnFirstLaunch()

    ListeningRecorder.shared.start()
    Analytics.start()
    return true
}
```

The generalisable lesson, and the sentence to say in the interview: *a responsibility must live on an object whose lifetime is at least as long as the events it is responsible for.* Progress recording outlives any screen, so it cannot live on one. This is the same reasoning that puts a background upload on a `URLSession` delegate rather than on a view controller.

When it writes

Four triggers, and the reason for each:

```

/// Writes down where an episode got to: when playback stops, when the engine
/// moves on to something else, when the app goes away, and once a minute
/// while it plays.

/// While playing, a write at least this often. Pausing and leaving are
/// caught as they happen; this is for what is not - the app ended by the
/// system mid-listen, or a crash, which otherwise lose the whole listen.
static let interval: TimeInterval = 60

```

Note that the minute is measured in **playback time, not wall-clock time**:

```

if isPlaying, abs(time - minuteMark) >= ListeningRecorder.interval {
    write("minute")
}

```

A timer would keep firing while paused, while seeking, while buffering. Using the position itself means a write happens exactly when a minute of *audio* has gone by, and pausing stops the clock for free. That is a small, real design insight.

Subscribing without hopping queues

```

/// Subscribed without hopping queues on purpose. The engine publishes on
/// the main thread already, and a write has to land before whatever the
/// caller does next - closing the player writes down the stop and then
/// clears what to reopen, and a write delivered a runloop later would
/// undo the clearing.
func start() {
    guard cancellables.isEmpty else { return }

    engine.sourcePublisher
        .sink { [weak self] in self?.sourceChanged(to: $0) }
        .store(in: &cancellables)
    // ...
}

```

`.receive(on: DispatchQueue.main)` looks harmless and would have introduced a genuine ordering bug: closing the player writes the final position and *then* clears the “what to reopen” slot. A write delivered one runloop later lands after the clear and resurrects it. **Ordering between a publisher and its caller is part of the contract**, and an unnecessary hop breaks it.

There is a test for exactly this:

```

func testStoppingWritesDownBeforeAnythingIsCleared()

```

The three-state problem: whose listening is this?

The hardest part of this class is not *when* to write but *whether this position is a listen at all*. Three different things can move the playhead:

What moved it	Is it a listen?	How the recorder learns
Audio playing	yes	<code>currentTimePublisher</code>
A person: scrubber, skip, lock screen, car	yes	<code>movedByHandPublisher</code>
Another device's position, adopted	no	<code>noteAdopted(position:)</code>

The third one caused a real, nasty bug. When the Mac has listened further, the phone moves its paused engine to the Mac's position so that pressing play carries on from there. That move looks exactly like listening — and the next write would stamp *the other device's position* with *this moment's date*, handing this device the account's newest listen over the device that actually listened. The account would then follow the wrong device forever.

```

/// Raised when the position was moved by another device and nobody here
/// has listened since. While it is up, nothing is written: what would be
/// written is somebody else's listening with this device's name and this
/// moment's date on it.
private var isAdopted = false

```

```

func noteAdopted(position: TimeInterval) {
    guard position > 0 else { return }
    self.position = position
    lastWritten = position
    minuteMark = position
    isAdopted = true
}

```

The first attempt at this fix compared seconds — “if the new position is within a second of what we adopted, do not write.” It failed, and the comment records why:

```

// Moved by another device and not listened to here since. The seek
// lands within a second of what was asked for, or does not land at
// all, and either way the number that comes back is not a listen -
// which is why this is a flag and not a comparison of seconds.
guard !isAdopted else { return }

```

State, not heuristics. A numeric comparison cannot distinguish “the seek landed near the target” from “the user happens to be listening near the target.” A flag can, because it records *provenance* rather than *value*.

The flag is lowered the instant this device does something real:

```

engine.isPlayingPublisher
    .sink { [weak self] playing in
        guard let self else { return }
        let wasPlaying = self.isPlaying
        self.isPlaying = playing
        // Playing here is what makes the position this device's own
        // again, whoever set it.
        if playing { self.isAdopted = false }
        if wasPlaying && !playing { self.write("paused") }
    }

```

```

/// Somebody here moved it: the scrubber, a skip button, the lock screen,
/// the car. That is this device deciding where to be, so whatever was
/// adopted from elsewhere is now this device's own position, and writing
/// resumes. Reached through the engine's own publisher, because a press
/// on the lock screen never passes through any screen of ours.
func noteMovedByHand(to position: TimeInterval) { ... }

```

That last comment is the key architectural insight of the whole chapter: **the engine is the only object every input passes through**, so it is the only object that can report provenance. The scrubber, the skip button, the lock screen’s `changePlaybackPositionCommand` and `CarPlay` all funnel into:

```

/// Fifteen seconds from a button, a drag of the scrubber, the same from
/// the lock screen or the car: all of them somebody here deciding where
/// to be, and all of them announced as such.
func moveByHand(to time: TimeInterval) {
    guard !isLive else { return }
    seek(to: time)
    movedByHand.send(max(0, time))
}

```

The idempotence guard

One more, and it is subtle enough to be worth a follow-up question:

```
// Nothing moved since the last write - paused, and then the app went
// to the background, or the player was closed. Writing again would
// only change the date, and the date is what decides which device
// listened last: a phone paused at 58 minutes and put away an hour
// later would claim the account's newest listen over the Mac that
// finished the episode in between.
if let lastWritten, abs(position - lastWritten) < 1 {
    minuteMark = position
    return
}
```

A write with an unchanged position is not a no-op, because **the timestamp is data**. Pausing, then backgrounding, then closing the player would write the same second three times with three increasing dates, and the last one wins the “which device listened most recently” comparison against a device that genuinely listened in between. Suppressing it is not an optimisation; it is correctness.

In an interview

This is your best “tell me about a hard bug” story. The full arc:

1. **Symptom** — listening in the car was never recorded, always resumed at the same second. No crash, no error, audio perfect.
2. **Why it was hard** — you cannot attach a debugger to a car. The evidence was one reproducible number.
3. **Root cause** — the object responsible for recording was owned by a screen that CarPlay never creates.
4. **Fix** — move the responsibility to an object with process lifetime, started from `didFinishLaunching`, subscribed to the engine rather than to any view.
5. **What it exposed** — once one object owned the writing, a second question appeared: *is this movement a listen at all?* Which produced the provenance flag, and the realisation that a numeric comparison could never answer it.
6. **How it is tested** — `ListeningRecorderTests` drives a fake engine with no view model anywhere in sight, which is the whole point:

```
/// Everything about when a listen is written down, against the engine alone –
/// no player view model, no screen. That is the point: the car starts the app
/// with no phone window, and the listen still has to be written.
```

```
/// The car: resumed at 57:45, a quarter of an hour of driving, the cable
/// pulled. iOS pauses the audio, and that pause is what gets written.
func testAListenInTheCarIsWrittenDownWhenTheCableIsPulled() {
    engine.play(.podcast(alarm), startingAt: 3_465)
    engine.advance(to: 4_365)
    engine.stopPlaying()

    XCTAssertEqual(progress.calls.last?.position ?? 0, 4_365, accuracy: 1)
}
```

The bug report, as a test, with the user’s actual numbers in it.

7. AVPlayer: five bugs and their shapes

Audio bugs are the best interview material this codebase has, because they are all *timing* bugs with no exception and no log line. Each one below is real, each has a one-line fix, and each generalises.

Bug 1 — The blip from the beginning

Symptom. Open an episode that was left at 57 minutes. For about half a second you hear the very beginning of the show, then it jumps to 57 minutes.

Cause. AVPlayer drops a seek issued before the item is `readyToPlay`. So the seek is parked and replayed from the status observer:

```
/// Where to jump once the new item is ready. Seeking a stream that has not  
/// finished loading is quietly dropped, so the request waits here instead.  
private var pendingSeek: TimeInterval?
```

That part was already right. What was wrong is that `play()` called `player.play()` unconditionally on the way out — so the file started at zero and the seek arrived a moment later.

```
// Not when there is a position to go to first. The status observer  
// plays once the seek has landed; playing here as well let the  
// opening second of the file out of the speaker before the seek  
// arrived - the blip from the beginning that the observer's own  
// comment says is fixed, and was not.  
if pendingSeek == nil {  
    player.play()  
}
```

```
/// Whether to start playing once that jump has landed. An episode opened  
/// at a position must not be heard from the beginning first, even for the  
/// half second it takes the seek to arrive.  
private var playAfterPendingSeek = false
```

And in the observer:

```
if item.status == .readyToPlay, let target = self.pendingSeek {  
    self.pendingSeek = nil  
    let resumeAfterwards = self.playAfterPendingSeek  
    self.playAfterPendingSeek = false  
  
    self.seek(to: target) {  
        // Only now. Playing first and seeking afterwards is  
        // what let the opening seconds of an episode out of  
        // the speaker before it jumped to where it was left.  
        guard resumeAfterwards else { return }  
        self.player?.play()  
        self.updateNowPlayingPlaybackState()  
    }  
}
```

The lesson worth saying out loud: a comment claiming a bug is fixed is not evidence that it is. The observer's own comment said the blip was handled; the fix was incomplete on a different line. Trust the symptom, not the annotation.

Bug 2 — The scrubber falling back

Symptom. Drag the scrubber to 40 minutes. It snaps back to 12 minutes for half a second, then jumps to 40.

Cause. `addPeriodicTimeObserver` keeps reporting the *old* position until the seek actually lands. Two pieces of state fix it:

```

/// Raised while a seek is in flight. The periodic observer keeps reporting
/// the old position until the seek lands, and letting that through drags
/// the slider back to where it was before the gesture.
private var isSeeking = false

/// Distinguishes a seek that finished from one a newer seek replaced, so
/// the older one cannot declare the newer one over.
private var seekGeneration = 0

// Shown straight away. Waiting for the seek to land leaves the slider
// sitting where it was for as long as the network takes.
currentTime = max(0, time)

seekGeneration += 1
let generation = seekGeneration
isSeeking = true

let seekingPlayer = player

player.seek(to: target, toleranceBefore: tolerance, toleranceAfter: tolerance) { [weak self, weak
↪ seekingPlayer] finished in
DispatchQueue.main.async {
    // The player is gone - stopped, or swapped for the downloaded
    // file. Nothing it was asked to do afterwards applies.
    guard let self, let seekingPlayer, self.player == seekingPlayer else { return }

    // Only the newest seek owns the state the seeks left behind.
    if generation == self.seekGeneration {
        self.isSeeking = false
        if finished {
            self.updateNowPlayingInfo()
        }
    }

    // Control comes back whichever seek was last, and outside that
    // guard on purpose.
    completion?()
}
}

```

Four separate ideas, and an interviewer can spend five minutes on any of them:

- **Optimistic UI.** `currentTime` is set to the target before the seek is issued, so the slider follows the finger.
- **The queue hop.** `AVFoundation` calls this handler on a queue of its own choosing, and everything inside it is main-thread state. This was missing for a long time and nothing ever visibly broke, which is what an unchecked concurrency assumption looks like right up until it does not.
- **A generation counter.** Drag across the bar and you issue many seeks. `AVPlayer` calls earlier completions with `finished == false`. Without the generation check, an old completion clears `isSeeking` while a newer seek is still in flight, and the stale-position guard opens at exactly the wrong moment. Same pattern as a request-id check in networking, and naming it that way in an interview lands well.
- **Two guards, because there are two questions.** This is the part worth telling, because it was got wrong twice in opposite directions.

The second act, which is the better half of the story

The first version called `completion?()` unconditionally, under a comment warning that an unfinished seek still has to hand control back. Then a review added the queue hop and, in the same change, moved the generation guard in front of `completion?()` — reasonable-looking, and it fixed a real thing: a seek issued against a player that has since been torn down must not start a *replacement* player.

It also reintroduced the hang the old comment was about. Press play on a restored episode — the seek is parked until the item is ready — then tap skip before the stream opens. The skip bumps the generation, the

parked seek's completion is the one that calls `player.play()`, and it is now dropped. Loaded, positioned, **silent**. Both are right about different things, and one counter cannot answer both:

Question	Answered by	If it fails
Does this completion still apply at all?	is <code>self.player</code> the same object?	drop everything
Does it still own <code>isSeeking</code> and <code>Now Playing</code> ?	the generation counter	skip the state, still hand back control

A superseded seek is stale about *where it was going*; it is not stale about the fact that somebody asked to start playing on arrival, and the newer seek has already moved where “there” is. So the identity guard wraps the whole body, the generation guard wraps only the state, and `completion?()` sits outside it. **Two different lifetimes wearing the same shape** — the same mistake as chapter 9's, three chapters away and in a completely different layer.

Bug 3 — Seek tolerance

```
// A second either way rather than an exact frame. Zero tolerance makes
// AVPlayer land precisely, which over a stream means waiting for data
// that has not arrived - going back is instant because it is already
// buffered, going forward stalls or is dropped. A second is nothing in
// a three hour show, and it is the difference between a scrubber that
// works and one that works sometimes.
let tolerance = CMTime(seconds: 1, preferredTimescale: 600)
```

Get the API right, because it is a common thing to get backwards in an interview: the convenience `seek(to:)` and `seek(to:completionHandler:)` do **not** mean zero tolerance — they are the efficient form and allow the player latitude. Sample accuracy is what you ask for *explicitly*, by passing `.zero` for both tolerances. So the choice here is not “stop using the expensive default”; it is picking a deliberate one-second window instead of either extreme.

Why it matters over a stream: an exact target means the player must have the surrounding data, and going forward into a buffer that has not arrived stalls where going back is instant. That asymmetry is the giveaway — *backwards instant, forwards hanging* is the shape of a buffering problem, not a seeking one. Worth adding honestly that tolerance was not the only cause in this app: waiting for `readyToPlay`, gating play on the seek's completion, and rejecting obsolete callbacks all fixed stalls too.

`preferredTimescale: 600` is the conventional choice because 600 is divisible by 24, 25, 30 and 60 — a safe common denominator for frame rates. It means ticks of $1/600$ s, about **1.67 ms**. Do not call that sub-millisecond, and do not call it sample accuracy: audio runs at 44 100 samples a second.

Bug 4 — Mutual recursion after an interruption

Symptom. Another app takes the audio session — a phone call, a navigation prompt. Press play afterwards: **crash**, stack overflow.

Cause. `resume()` saw the item had failed and called `play()`. `play()` saw the same source with a player still attached and called `resume()`. Each called the other until the stack ran out.

```

/// Starts playback of `source`. Re-selecting what is already loaded just
/// resumes - unless what is loaded has failed, in which case there is
/// nothing to resume and it is built again.
///
/// A failed item used to take the shortcut too: resume() saw the failure
/// and called play(), play() saw the same source with a player still
/// attached and called resume(), and the two called each other until the
/// stack ran out. A press on play after another app had taken the audio
/// session was a crash rather than a recovery.
func play(_ source: PlaybackSource, startingAt position: TimeInterval? = nil) {
    let isLoadingAndWell = player != nil && player?.currentItem?.status != .failed

    if source.isSameContent(as: self.source), isLoadingAndWell {
        ...
    }
}

```

The fix is one clause: `player != nil` is not the same question as “*is there something here I can resume?*” A failed `AVPlayerItem` is non-nil and cannot be resumed — it must be rebuilt. Two functions that delegate to each other need a condition that is **strictly narrowing** on at least one side, or they recurse.

The interruption handling itself is textbook, and worth knowing cold:

```

switch type {
case .began:
    pause()
case .ended:
    guard let rawOptions = info[AVAudioSessionInterruptionOptionKey] as? UInt else { return }
    if AVAudioSession.InterruptionOptions(rawValue: rawOptions).contains(.shouldResume) {
        resume()
    }
@unknown default:
    break
}

```

`.shouldResume` is a *hint from the system*, not a guarantee, and resuming without checking it is the classic mistake — it makes your app start talking over the thing that interrupted it.

Bug 5 — Zero is not a position

```

timeObserverToken = player.addPeriodicTimeObserver(forInterval: interval, queue: .main) { [weak self] time in
    guard let self else { return }
    // Duration still updates: only the position is stale mid-seek.
    //
    // A non-finite time is not a position of zero, it is no position at
    // all - what a player reports once its item has failed or been torn
    // down. Writing it as zero threw away the one number needed to carry
    // on from where the interruption happened.
    if !self.isSeeking, time.seconds.isFinite {
        self.currentTime = time.seconds
    }
}

```

`CMTime` has `.indefinite` and `.invalid`; `.seconds` on either is `NaN`. `NaN` coerced into a `TimeInterval` and written to the store destroys exactly the number needed to recover. **Absence and zero are different values**, and conflating them is one of the most reliably expensive mistakes in this domain — it is the same class as `Int (someOptional ?? 0)` in a totals column.

The tidying that is not optional

```
private func teardownPlayer() {
    if let token = timeObserverToken {
        player?.removeTimeObserver(token)
        timeObserverToken = nil
    }
    if let endObserver {
        NotificationCenter.default.removeObserver(endObserver)
        self.endObserver = nil
    }
    if let metadataOutput {
        player?.currentItem?.remove(metadataOutput)
        self.metadataOutput = nil
    }
    isSeeking = false
    timeControlObservation?.invalidate()
    timeControlObservation = nil
    itemStatusObservation?.invalidate()
    itemStatusObservation = nil
    player?.pause()
    player = nil
}
```

Know this list, because it is a standard interview question in audio/video roles. A periodic time observer that is not removed **before** the player is released is a documented crash, not a leak. `NSKeyValueObservation` must be `invalidate()`d. Block-based `NotificationCenter` observers must be removed by token. And `isSeeking = false` is in there because a teardown mid-seek would otherwise leave the flag stuck and freeze the scrubber on the next item.

In an interview

Question: “What is hard about building an audio player?”

Answer: “Nothing about it is hard until you care about the *position*. Play/pause is twenty lines. Everything expensive comes from the fact that `AVPlayer` is asynchronous in five places that do not agree with each other: the item is not ready when you get it, seeks are dropped before it is ready, the periodic observer reports stale positions while a seek is in flight, seeks you issued are cancelled by seeks you issue next, and a failed item looks exactly like a loaded one until you ask its status. Every bug we had was one of those five, and every fix was one piece of state: a pending seek, a seeking flag, a generation counter, a status check.”

Follow-up: “How do you test that?” — mostly you do not, at the `AVPlayer` level; you test the *decisions* against a protocol (chapter 13). But there is one test in the suite that does drive a real `AVPlayer`, over a generated silent file, precisely because the end-of-file behaviour is the one thing a fake cannot tell you:

```
final class PlaybackReplayTests: XCTestCase {
    func testActualPlayerCanResumeAndExplicitlyReplayAfterEOF() throws
```

Knowing *where* the fake stops being enough is the senior part of the answer.

8. Stream metadata, and giving it a lifetime

The problem

A live radio stream announces what is playing. Two things go wrong with that, and both were real user reports.

First: the screen showed 14 - artist. Not a song — a file number and the *name of a metadata field* that the encoder sent because nobody filled the tags in.

Second, and worse: a song caught at 07:55 was still on screen at 09:30. The station names a song when it starts and says *nothing at all* when the programme begins. With no end to an announcement, the last song before eight o'clock sat on the screen through the whole show — **and went into every bookmark made during it, as if that were what was playing**. A metadata bug had become a data-corruption bug.

The code: one parser for two transports

```
/// A track announced by the stream itself. Shoutcast/Icecast send one string,
/// almost always "Artist - Title", and HLS streams send the same thing as timed
/// metadata - so one parser covers both.
struct LiveTrack: Equatable {
    let artist: String?
    let title: String
}
```

A failable initialiser that returns nil for anything not worth showing:

```
/// The words an encoder sends when nobody filled the tags in: the name of
/// the field instead of its value. "14 - artist" is a file number and a
/// placeholder, not a song, and putting it on the screen as one is worse
/// than showing nothing - the screen already says what is playing.
private static let placeholders: Set<String> = [
    "artist", "title", "artist - title", "unknown", "unknown artist",
    "nepoznato", "n/a", "na", "-", "---",
]

init?(raw: String) {
    let cleaned = raw.trimmingCharacters(in: .whitespacesAndNewlines)
    guard !cleaned.isEmpty, cleaned.count < 200 else { return nil }
    guard !cleaned.lowercased().hasPrefix("http") else { return nil }

    let stationNames = ["daskoimladja", "dasko i mladja", "daško i mlađa", "radio"]
    if stationNames.contains(cleaned.lowercased()) { return nil }
    if LiveTrack.placeholders.contains(cleaned.lowercased()) { return nil }

    if let separator = cleaned.range(of: " - ") ?? cleaned.range(of: " — ") {
        let artist = String(cleaned[..<separator.lowerBound]).trimmingCharacters(in: .whitespaces)
        let title = String(cleaned[separator.upperBound...]).trimmingCharacters(in: .whitespaces)

        // Either half being a placeholder condemns the whole announcement:
        // whichever side the real value was meant to be on, it is not
        // there, and the half that is left is a number or a station name.
        if LiveTrack.placeholders.contains(artist.lowercased()) { return nil }
        if LiveTrack.placeholders.contains(title.lowercased()) { return nil }

        if !artist.isEmpty && !title.isEmpty {
            self.artist = artist
            self.title = title
            return
        }
    }

    self.artist = nil
    self.title = cleaned
}
```

Three things to point at:

- **The failable initialiser is the design.** Validation lives at the boundary, in the type's own constructor, so a LiveTrack that exists is a LiveTrack worth drawing. No `isValid` flag for callers to forget.
- **"Either half condemns the whole."** 14 - artist has a plausible left side. Salvaging it would put a file number on the lock screen. When part of a record is known-bad, the record is bad.
- **Both dash characters.** " - " and " – ", hyphen and en dash, because encoders send both. And the separator is searched *with spaces around it*, so AC-DC survives — a real hazard in this exact parser.

There is a test asserting a numeric title from a real artist survives, which is the boundary case the placeholder list could easily have eaten:

```
func testANumericTitleFromARealArtistSurvives()
```

Giving an announcement an end

This is the more interesting half, because the bug is about *absence of data*.

```
/// How long a song announced by the station stands on its own.
///
/// The station names a song when it starts and says nothing when the
/// programme begins. With no end to it, the last song before eight stayed
/// on the screen through the whole show - and went into every bookmark
/// made during it as if that were what was playing. Ten minutes is longer
/// than nearly every song and far shorter than a programme.
static let liveTrackLifetime: TimeInterval = 10 * 60
private var liveTrackExpiry: DispatchWorkItem?
```

```
/// One announcement, and an end to it - see liveTrackLifetime.
private func setLiveTrack(_ track: LiveTrack?) {
    liveTrackExpiry?.cancel()
    liveTrackExpiry = nil
    liveTrack = track
    guard track != nil else { return }

    let expiry = DispatchWorkItem { [weak self] in
        guard let self, self.isLive else { return }
        self.liveTrack = nil
        self.updateNowPlayingInfo()
    }
    liveTrackExpiry = expiry
    DispatchQueue.main.asyncAfter(deadline: .now() + PlaybackEngine.liveTrackLifetime,
                                  execute: expiry)
}
```

The generalisable idea, and the sentence that makes this a senior answer: *a push feed that only reports starts needs the receiver to supply the ends*. The protocol gives you “song began”; it never gives you “song ended”. If your model has no expiry, your UI asserts something false for an unbounded time. This is the same reasoning as a TTL on a cache entry, a lease in a distributed system, or a presence timeout in a chat client.

The `DispatchWorkItem` is the right tool rather than a `Timer` because it is cancellable by identity — every new announcement cancels the previous expiry and schedules its own, so there is never more than one in flight and no bookkeeping about which timer belongs to which track.

Clearing versus not-clearing

```
// An empty announcement between songs should clear the label rather than
// leave the previous track sitting there as if it were still playing.
guard announced != nil || sawAnyItem else { return }
guard announced != liveTrack else { return }
```

`sawAnyItem` distinguishes “the stream said something and none of it parsed” — clear the label — from “the callback

fred with nothing in it” — leave it alone. Two different absences, two different responses. Same theme as chapter 7’s bug 5.

A type-checker war story

```
// Written as a plain loop on purpose. The same thing as a chain of
// flatMap/filter/compactMap made the type checker give up on this
// expression - AVMetadataItem's overloads leave it too much to infer.
var announced: LiveTrack?
var sawAnyItem = false

for group in groups {
    for item in group.items {
        sawAnyItem = true
        guard isTitleMetadata(item), let raw = item.stringValue else { continue }
        if let track = LiveTrack(raw: raw) { announced = track }
    }
}
```

Worth having as a small opinion: “the expression was too complex to be solved in reasonable time” is not a compiler bug to be worked around with type annotations until it goes away — it is a signal that the expression is doing too much. A loop that anyone can read in four seconds is not a regression from a functional chain that takes the compiler ninety seconds and a reader thirty.

And the metadata reaches the lock screen

```
// While live, the lock screen and the car should show the song rather
// than "Radio uživo", which they already know from the live badge.
let displayTitle = source.isLive ? (liveTrack?.title ?? source.title) : source.title
```

```
var info: [String: Any] = [
    MPMediaItemPropertyTitle: displayTitle,
    MPMediaItemPropertyArtist: displayArtist,
    MPNowPlayingInfoPropertyIsLiveStream: source.isLive,
    MPNowPlayingInfoPropertyElapsedPlaybackTime: currentTime,
    MPNowPlayingInfoPropertyPlaybackRate: isPlaying ? 1.0 : 0.0
]

if !source.isLive, duration > 0, duration.isFinite {
    info[MPMediaItemPropertyPlaybackDuration] = duration
}
```

MPNowPlayingInfoPropertyIsLiveStream is the flag that makes the system hide the scrubber for live audio, and omitting PlaybackDuration for a live source is the other half of the same statement. Small, specific platform knowledge — exactly the kind that separates “I have used AVFoundation” from “I have shipped with it.”

In an interview

Question: “How do you handle data from a source you do not control?”

Answer: “Validate at the boundary, in a failable initialiser, so the rest of the app only ever sees values worth using. Then ask what the feed does *not* tell you — ours announced song starts and never song ends, so the last song before the morning show stayed on screen for two hours and ended up copied into every bookmark made during it. The fix was to give every announcement a ten-minute lifetime on our side. The general rule is that a push feed that only reports starts makes the receiver responsible for the ends.”

9. Domain rules as values

The problem

“When is an episode finished?” sounds like a one-liner and is not. The wrong answer has two visible failure modes: an episode goes grey while there is still show left, or it never goes grey because the listener stopped during the closing credits.

There is a second, different question that *looks* the same and is not: “should pressing play start this episode over?” Conflating them was a real, recent bug.

The code

Two conditions, and whichever lies later wins:

```
/// How much of an episode has to be behind you before it counts as heard.
/// One of the two conditions; see endOfShow for the other.
static let playedFraction = 0.95

/// The finish line: both conditions, so whichever of the two lies later.
/// Ninety-five percent of the running time has to be behind you *and* the
/// closing credits have to have started.
///
/// Which one binds depends on the length. On a three-hour Alarm the
/// credits are the later line by eight minutes, so they are what settles
/// it; on a five-minute episode ninety-five percent falls after the
/// credits begin, and it is the percentage that settles it. Requiring
/// both means an episode is never called heard with a stretch of show
/// still in front of it.
var endOfShow: TimeInterval {
    let duration = durationInSeconds
    guard duration > 0 else { return 0 }
    return max(0, max(duration * Podcast.playedFraction, duration - outro))
}
```

Work the arithmetic, because an interviewer will:

- **Three-hour episode, 20 s of credits.** 95 % = 10 260 s. Credits start at 10 780 s. `max` → **10 780**. The credits bind, and they are 8½ minutes later than the percentage. A percentage alone would mark the episode heard with 8½ minutes of actual show still to go.
- **Five-minute episode, 20 s of credits.** 95 % = 285 s. Credits start at 280 s. `max` → **285**. Now the percentage binds. A credits-only rule would mark it heard 5 s too early.

Neither rule alone is right at both scales. `max` of the two is right at both. That is the whole answer, and it is a genuinely good one.

The outro figure itself is layered, with a deliberate treatment of zero:

```
/// The closing credits, in seconds: what the backend measured for this
/// episode, or the show's figure when it has not measured one. Backend
/// zero is unmeasured, not measured-as-none, so it does not overrule a
/// show the app knows about.
var outro: TimeInterval {
    if let outroSeconds, outroSeconds > 0 { return outroSeconds }
    return show.outroSeconds
}
```

Three-tier fallback — **episode** → **show** → **nothing** — and 0 read as “absent” rather than “measured as none,” because the backend column defaults to zero and an old cached row has no value at all. Same distinction as chapter 7’s bug 5, in a completely different layer. *Absence and zero are different values* is the recurring theme of this codebase.

Two lines, not one

The recent bug: pausing anywhere in the last 5 % offered “Play from the start” instead of carrying on. On a three-hour show that means the button changed meaning 8½ minutes before the end, and pressing it threw away three hours.

The cause was using one predicate for two questions. The fix is a second, narrower one:

```
/// The very last second of the file, which is the only place where
/// starting over is the obvious thing to offer.
///
/// Deliberately not the same line as `hasReachedEnd`. That one asks
/// whether the episode counts as heard, and answers yes with the closing
/// credits still running - pausing there and coming back should carry on
/// from where it stopped, not rewind three hours.
func isAtVeryEnd(at position: TimeInterval, duration: TimeInterval? = nil) -> Bool {
    let length = duration.flatMap { $0.isFinite && $0 > 0 ? $0 : nil } ?? durationInSeconds
    guard position.isFinite, position > 0, length.isFinite, length > 0 else { return false }
    return position >= length - 1
}
```

```
/// Only at the very end of the file. An episode paused in the closing
/// credits counts as heard, but it is still a pause - it carries on.
var offersReplay: Bool {
    guard !isPlaying, !isBuffering, !isLive, let podcast else { return false }
    return podcast.isAtVeryEnd(at: currentTime, duration: duration)
}
```

The lesson: two questions that agree on most inputs are still two questions. “Counts as heard” (a *record*, sticky, syncs, greys the row out) and “is at the end” (a *transport state*, momentary, local) share a shape and nothing else. The test names the distinction:

```
/// The final ten seconds are past the show-end threshold. It is
/// still a pause: pressing play again carries on, it does not rewind.
func testPausingInTheClosingCreditsCarriesOn() {
    // ... advance to 10_790 of 10_800, stop playing ...
    XCTAssertTrue(alarm.hasReachedEnd(at: 10_790))
    XCTAssertFalse(player.offersReplay)
    XCTAssertEqual(player.playButtonSymbol, "play.fill")
    player.togglePlayPause()
    XCTAssertEqual(player.currentTime, 10_790, accuracy: 0.5)
}
```

That test first went in at **10 500**, and a review caught it: 10 500 is inside the last five percent, which is what the bug report said, but a three-hour episode’s show-end line is at 10 780, so the position never crossed the threshold the test’s name claims. It passed, and it passed for the wrong reason. At 10 790 it is genuinely past the end of the show — hence the added `hasReachedEnd` assertion, which pins the precondition instead of assuming it. **A test whose name describes a boundary should assert that it is on the far side of it.**

Where to resume, which is a third question

```

/// Below this, a saved position is not worth returning to - the first
/// seconds of an episode are quicker to hear again than to think about.
static let resumeFloor: TimeInterval = 20

/// Where pressing play should pick this episode up, or nil to start at
/// the beginning.
///
/// Where the listen got to is what settles this, not whether the episode
/// was ever heard through. `isPlayed` is sticky and says what happened
/// once; it says nothing about where anybody is now.
var resumePosition: TimeInterval? {
    guard playedPosition > Podcast.resumeFloor else { return nil }
    guard !hasReachedEnd else { return nil }
    // A few seconds back, for the same reason a bookmark takes a few: you
    // stopped listening slightly before you stopped playing.
    return max(0, playedPosition - 3)
}

```

That first guard used to read `!isPlayed`, `playedPosition > ...`, and the bug it caused is the third member of this chapter's family. An episode heard through months ago, started again in the car and paused twenty-five minutes in, has `isPlayed == true` and `playedPosition == 1500`. Asking the flag threw the twenty-five minutes away and started the show over — and the same flag in `lastListened`'s fetch predicate dropped that episode out of the car's *Carry on* row entirely, so the car offered something else while the thing you were actually in the middle of was invisible.

Three questions, not one, and the third only surfaced when a real person re-listened to something in a car:

Question	Answered by	Sticky?
Was it ever heard through?	<code>isPlayed</code>	yes
Has this position passed the end of the show?	<code>hasReachedEnd</code>	no
Is it at the very last second?	<code>isAtVeryEnd</code>	no
Where does it carry on from?	<code>playedPosition + hasReachedEnd</code>	no

Only the first is sticky, and only the first is a record of history. The other three are all questions about *this position, right now*, and every bug in this family came from letting the sticky one answer one of them. Both later bugs were the same mistake at different scales: the first conflated two of these questions, the second conflated all three.

Both constants encode something about *people* rather than about audio. The 20-second floor: below that, the thinking costs more than the listening. The 3-second rewind: **you stopped listening slightly before you stopped playing** — by the time your hand reaches the button, you have missed a sentence. The same figure is used for bookmarks, which is why a bookmark points at the joke rather than at the silence after it.

And a fourth: what to draw

```

/// How far through the show a listen got, as a fraction - for the line
/// under an episode in a list.
///
/// Nil for an episode that has not been started and for one whose position
/// is at the end: an empty line and a full line each say nothing, and
/// drawing them puts a rule under every row in the list for no reason.
var listeningProgress: Double? {
    guard !hasReachedEnd, playedPosition > 0 else { return nil }
    let end = endOfShow
    guard end > 0 else { return nil }
    // A minimum, so a minute into a three-hour episode is still visible as
    // something rather than as a line that was never drawn.
    return min(max(playedPosition / end, 0.02), 1)
}

```

`Double?` rather than `Double`, so “no bar” is a value the type carries rather than a rule every call site has to

remember. The 2 % floor is an honest compromise: a minute into three hours is 0.5 % and rounds to an invisible line, so the bar lies slightly in order to communicate. Being able to defend a deliberate inaccuracy in a UI is a nice thing to have in your pocket.

The text beside it is tied to the same nil:

```
/// Nil in exactly the cases where the bar is nil, so the two appear and
/// disappear together.
var remainingDescription: String? {
    guard listeningProgress != nil else { return nil }
}
```

Derived optionality — one predicate, two outputs — so the bar and the label cannot disagree. Compare with chapter 5's "one rule, two read paths": the same instinct, applied to presentation.

Why any of this is testable

All of it is on the Podcast **struct**, computed from stored values, with no dependency on the store, the network, or the player. ListeningRulesTests has 25 tests and needs no fixtures beyond a struct literal:

```
/// Three hours with twenty seconds of credits: the show ends at 10 780.
private func makeEpisode(title: String) -> Podcast {
    var podcast = Podcast(show: .alarmSaDaskomIMladjom)
    podcast.itunesDuration = "3:00:00"
    podcast.outroSeconds = 20
    ...
}
```

This is the payoff of putting domain rules on value types, and it is a strong thing to say in an interview: the rules that are hardest to get right are exactly the ones that are cheapest to test, *provided they do not depend on anything*. The moment hasReachedEnd needs a repository, its test needs a database.

In an interview

Question: "Where do you put business logic?"

Answer: "On the value type, as computed properties, when it is a pure function of data the type already holds. Ours answers four questions about an episode — is it finished, is it at the very end, where should it resume, what bar should the row draw — and none of them touches the store or the network, so twenty-five tests run in milliseconds against struct literals. The interesting part was discovering that two of those questions, which agree on almost every input, are genuinely different: 'counts as heard' is a sticky record that syncs, 'is at the end' is a momentary transport state. We shipped them as one predicate and it meant pausing in the closing credits offered to restart a three-hour show."

Part III — Four faces, one codebase

10. iPhone, iPad, Mac and CarPlay from one target

The claim to be careful about

“It runs on iPhone, iPad, Mac and CarPlay” sounds like four builds. It is one target, one binary per platform family, and — this is the number worth quoting — `#if targetEnvironment(macCatalyst)` appears in six files out of forty-seven.

AlarmDMPApp.swift	1	(close the window, keep playing)
AppDelegate.swift	1	(no audio session on the Mac)
PlaybackEngine.swift	5	(session, interruptions)
CarPlaySceneDelegate.swift	1	(the whole file is compiled out)
SettingsView.swift	5	(no mail composer, no share sheet)
SupportView.swift	2	

Everything else adapts at runtime. That ratio is the interview answer: **the compile-time switches are for APIs that do not exist on the other platform; everything about *shape* is a runtime decision.** Mixing those two up is what produces a codebase with `#if` in every view.

Compile-time: when the API is absent

The audio session is the clean case. `AVAudioSession` is an iOS API, and the Catalyst build has no single session to claim — the system mixes applications itself — so the three calls that matter are wrapped once rather than guarded at every call site. (Scope this claim carefully if pressed: it is this app’s platform policy, not a promise that a Mac never sees a route change. And note that an iOS build running on Apple silicon is a third case again, which is why the *interaction* checks below are runtime ones.)

```
// AVAudioSession is an iOS idea. On the Mac there is no single session to
// claim, no category to declare and nothing to be interrupted by - the
// system mixes applications itself. So the three calls that matter are
// wrapped here rather than guarded at each of their call sites, and on the
// Mac they simply do nothing.

private func activateSession() {
    #if !targetEnvironment(macCatalyst)
    do {
        try AVAudioSession.sharedInstance().setActive(true)
    } catch {
        AppLog.write(.player, "Audio session activation failed: \(error.localizedDescription)")
    }
    #endif
}
```

Five `#ifs` in one file, all of them at the bottom of the stack, and zero above it. Callers say `activateSession()` and do not know there is a platform question.

CarPlay is even simpler — the framework does not exist on the Mac, so the whole file goes:

```
// CarPlay does not exist on the Mac, and neither does the framework - the
// whole file is compiled out there rather than guarded piece by piece.
#if !targetEnvironment(macCatalyst)
```

Runtime: when the shape differs

Two mechanisms, used for different questions.

Size class, for how much room there is:

```
/// How wide the content is allowed to get before it stops following the
/// window. A list of episode titles set to the full width of a Mac window
/// is a line of text with a hundred points of subject and eight hundred of
/// nothing, and an eye has to travel all of it. Everything that scrolls
/// keeps to this, and so does the mini player, so the two line up.
static func contentWidth(for widthClass: UserInterfaceSizeClass?) -> CGFloat {
    widthClass == .regular ? 900 : .infinity
}
```

A typographic measure, not a device check — which is why it is correct for an iPad in Split View, a resized Mac window and a landscape iPhone without any of them being enumerated.

Interaction model, for how input works:

```
/// Swipe and long press carry the same actions. Swipe is fast for anyone who
/// knows it is there; the context menu is how everyone else finds it, which is
/// the same pairing Apple's own Podcasts app uses.
struct EpisodeRowActions: ViewModifier {

    static var showsInlineActions: Bool {
        ProcessInfo.processInfo.isMacCatalystApp || ProcessInfo.processInfo.isiOSAppOnMac
    }
}
```

Note this is `ProcessInfo`, **not** `#if`. It has to be, because `isiOSAppOnMac` — an unmodified iOS build running on Apple silicon — is a condition that only exists at runtime; there is no compile-time symbol for it. The distinction is a genuinely good piece of trivia to have: `#if targetEnvironment(macCatalyst)` is false in that case, and a codebase that only guards with `#if` ships a Mac experience with swipe gestures nobody can perform.

And the action set is shared, so the two presentations cannot drift:

```
@ViewBuilder
func body(content: Content) -> some View {
    if Self.showsInlineActions {
        // Siblings of the tappable row: these buttons must not start playback.
        HStack(spacing: 12) {
            content
            inlineActions
        }
    } else {
        touchActions(content: content)
    }
}
```

The comment on the `HStack` is a real bug's headstone: put a button *inside* a tappable row and you get both the button's action and the row's.

The Mac window that must not quit

```

#if targetEnvironment(macCatalyst)
// Closing the window no longer quits the app, which is the point: the
// radio carries on, driven from Now Playing in the menu bar and the
// media keys, and a click on the Dock icon brings the window back.
//
// Supporting that is what makes a second window possible, and one
// player has no use for two. File > New Window goes.
.commands {
    CommandGroup(replacing: .newItem) {}
}
#endif

```

Paired with the Info.plist:

```

<!-- True so the Mac keeps running with no window open: a Catalyst
     app that supports only one window quits when it is closed,
     and the radio with it. -->
<key>UIApplicationSupportsMultipleScenes</key>
<true/>

```

This is a two-part fix where the parts are in different languages, and that is what makes it memorable. A Catalyst app declaring single-scene support terminates when its last window closes — so closing the red X killed the radio, which is not what a Mac user means by closing a window. Turning on multiple scenes fixes it, and then the *consequence* of turning it on is that **File > New Window** appears, which for a single-player app is a way to get two of something there can only be one of. So the menu item is removed. *Enabling a capability to get a side effect, then suppressing the capability's own affordance.*

CarPlay: a second UI with the same rules

The design rule for CarPlay in this app is one sentence, and it is worth learning verbatim because it is the answer to “how do you avoid duplicating logic across surfaces?”:

The car has no rules of its own.

```

item.handler = { [weak self] _, completion in
    guard let self else { completion(); return }

    if case .podcast(let playing) = self.engine.source, playing.id == podcast.id {
        // Already loaded: carry on rather than open it again, and
        // never stop it - see openRadio.
        if !self.engine.isPlaying { self.engine.resume() }
    } else {
        // Where the app would have started it. The rule lives in the
        // repository precisely so the car cannot have its own.
        self.engine.play(
            .podcast(podcast),
            startingAt: PodcastRepository.shared.resumePosition(for: podcast.id)
        )
    }

    self.pushNowPlaying()
    completion()
}

```

The repository comment says the same thing from the other side:

```

/// Every way into playback has to ask this - the phone, the car, the lock
/// screen - or the rules about where a listen resumes only hold on the
/// screen they were written for. That is exactly how starting an episode
/// from CarPlay went back to the beginning while the same episode on the
/// phone carried on.
func resumePosition(for id: UUID) -> TimeInterval? {

```

That was a real bug and it is the second-best story in the guide after the CarPlay recorder: the resume rule existed, it was correct, and it lived in the view model. The car did not go through the view model, so the car did not get the rule. **A rule that lives on a caller is not a rule; it is a habit.**

Three CarPlay details worth knowing

One tap to audio. CarPlay's guidelines want the shortest possible path to playing audio while driving, so there is no detail screen:

```

/// One tap plays the episode - no intermediate menu. CarPlay guidelines want
/// the shortest possible path to audio while driving.

```

A failure with no visible cause, retried. A blank CarPlay screen with audio still playing means the root template never landed, and CarPlay says nothing about it:

```

/// A blank CarPlay screen with the audio still playing means the root
/// template never landed - the scene is connected and nothing was ever
/// handed to it. It is the one failure here with no visible cause, since
/// CarPlay says nothing and the app carries on, so the result is asked for
/// and a failure is tried again rather than left as an empty screen.
private func setRoot(_ template: CPTemplate, retriesLeft: Int) {
    guard let interfaceController else { return }

    interfaceController.setRootTemplate(template, animated: true) { [weak self] done, error in
        guard !done, retriesLeft > 0 else { return }
        DispatchQueue.main.asyncAfter(deadline: .now() + 0.5) {
            self?.setRoot(template, retriesLeft: retriesLeft - 1)
        }
    }
}

```

Bounded retry with backoff on an API whose completion handler most code ignores. *Taking the completion handler seriously on an API that “cannot fail”* is a good habit to be able to point at.

Rebuild the sections, not the template.

```

/// Kept so the sections can be rebuilt in place. Replacing the root
/// template instead would throw away wherever the person had navigated
/// to, which in a car is worse than a stale row.
private var radioTemplate: CPListTemplate?

```

The cost model is different in a car. Losing someone's navigation position while they are driving is worse than showing a row that is five seconds out of date — a judgement about the *context of use*, not about the API.

Designing for a glance. The progress bar on a CarPlay row:

```

// CarPlay draws the bar itself, under the row, which is the one place
// on that screen with room to spare. Full for a finished episode: from
// a car seat a full bar says "heard" faster than any mark could.
item.playbackProgress = CGFloat(podcast.listeningProgress ?? (podcast.isPlayed ? 1 : 0))

```

Recall from chapter 9 that `listeningProgress` is deliberately `nil` for a finished episode, because a full bar in a phone list says nothing. In a car it says everything, because there is no time to read. **The same data, presented oppositely, because the reading conditions are opposite.**

And the two cuts of the same broadcast:

```
// The two cuts of one day have the same title, so on this screen they
// were two identical rows. The one without music is marked, where the
// eye ends a row; the one with music is the show as it went out and
// is left plain, as it is everywhere else.
if marksCut && !podcast.isWithMusic {
    item.setAccessoryImage(Self.withoutMusicMark)
}
```

Only the *exception* is marked, and only where an ambiguity actually exists — `showsInBothCuts` computes which shows publish both, so shows that publish one cut get no mark at all. Marking both would be symmetrical and worse: two marks never look like a pair, and a mark that is always present carries no information.

In an interview

Question: “How do you share code across iOS and macOS?”

Answer: “One target, and a rule about which kind of difference gets which mechanism. `#if` only where the API genuinely does not exist — for us that is `AVAudioSession` and `CarPlay`, six files out of forty-seven, all of them at the bottom of the stack so nothing above knows there is a question. Everything about shape is runtime: size class for how much room there is, `ProcessInfo` for the interaction model. `ProcessInfo` rather than `#if` specifically because an iOS build running on Apple silicon is not `macCatalyst` at compile time, so a codebase that only uses `#if` ships swipe actions to a Mac.”

The stronger follow-up answer, if they ask about `CarPlay`: “`CarPlay` is not a port, it is a second head on the same engine, and the rule is that it has no rules of its own. Every decision about where an episode resumes lives in the repository, because the first version had it in the view model — and the car does not go through the view model, so starting an episode in the car went back to the beginning while the same episode on the phone carried on.”

11. Localization, and forcing a default

The problem

An app written in Serbian, for a Serbian radio station, opened by listeners in Serbia, **came up in English**.

The cause is ordinary and worth knowing precisely: iOS does not pick your app's language from the region or from the user's country. It walks the device's *preferred language list* and takes the first entry your bundle supports. Practically every phone lists English somewhere — often above the local language, because that is how people set them up.

The code

```
/// Which language the app speaks.
///
/// iOS picks from the phone's preferred languages, and almost every phone
/// lists English somewhere. So an app written in Serbian, for a Serbian radio
/// station, opened in Serbia, came up in English - an answer nobody asked
/// for, since the episodes, the shows and the station are all Serbian anyway.
/// English is here for the rare person who wants it, not as the default for
/// everyone whose phone happens to be set up in US English.
///
/// The way to say so is the one iOS uses itself: the per-app language lives in
/// the app's own defaults under `AppleLanguages`, and the Language row in
/// Settings reads and writes exactly that. Writing it once, on the first
/// launch, is the same as the person having chosen Serbian there - and their
/// own choice afterwards is never touched again.
enum AppLanguage {

    static let serbian = "sr-Latn"

    private enum Key {
        /// Apple's own, in this app's defaults domain.
        static let languages = "AppleLanguages"
        /// Ours, so that the line above is written exactly once ever.
        static let defaulted = "languageDefaultedToSerbian"
    }

    static func applyDefaultOnFirstLaunch(_ defaults: UserDefaults = .standard) {
        guard !defaults.bool(forKey: Key.defaulted) else { return }
        defaults.set(true, forKey: Key.defaulted)
        defaults.set([serbian, "en"], forKey: Key.languages)
        speakSerbianForThisLaunch()
    }
}
```

Three properties make this defensible rather than hostile:

- **It writes the same key the system writes.** iOS's per-app Language setting is `AppleLanguages` in the app's own defaults domain, so this leaves the user's own control working rather than fighting it. Be honest about what it is not: writing that key yourself is a compatibility move, not a documented language-switching API, so it is worth re-testing on a clean install with each OS rather than assuming it keeps working.
- **It runs once, ever.** Guarded by a key of our own — so if the user later picks English in Settings, that overwrites `AppleLanguages` and this code never writes again.
- **It is a *default*, not a *lock*.** The English localization is complete and one tap away.

The launch-ordering problem, and a failed fix

Writing `AppleLanguages` takes effect **at the next launch**. The bundle's language is resolved before any of your code runs. So the very first launch — the only one where it matters for a first impression — was still English.

The obvious fix is the one everyone finds on Stack Overflow: swizzle `Bundle.main` with a subclass that overrides `localizedString(forKey:...)`. It was tried, and the comment records the experiment and its result:

```

/// All it does is raise a flag. Bundle.main was tried first, on the
/// theory that every lookup goes through it; a debug line either side of
/// the swap answered "Live radio" both times, so nothing the app draws
/// comes from there. What SwiftUI does use is the locale in the
/// environment, which is what the flag below feeds.
private static func speakSerbianForThisLaunch() {
    guard Bundle.main.preferredLocalizations.first != serbian else { return }
    isForcingThisLaunch = true
}

```

```

/// What the views are given. SwiftUI resolves a literal against the
/// locale in the environment, and that is the one lever that reaches the
/// text on screen.
static var localeForThisLaunch: Locale {
    isForcingThisLaunch ? Locale(identifier: serbian) : .current
}

```

```

WindowGroup {
    rootView()
    // Only ever different from the phone's on the first launch -
    // see AppLanguage.
    .environment(\.locale, AppLanguage.localeForThisLaunch)
}

```

Tell this one as a debugging story, not as a solution. The bundle swap is the popular answer and it did nothing, and the way that was established was a log line either side of the swap printing the same English string both times. Text("Live radio") in SwiftUI is a LocalizedStringKey, and SwiftUI resolves it against \.locale in the environment — not through Bundle.main.localizedString. One environment value reaches what the bundle swap could not. *The fix is smaller than the workaround, and it was found by instrumenting the workaround instead of trusting it.*

String Catalogs, and String versus LocalizedStringKey

Source language sr-Latn, 147 keys, English for each. The trap that produces most localization bugs in Swift is the type distinction:

- Text("Pusti") — the parameter is LocalizedStringKey, so it localizes.
- Text(someString) — a String overload, so it does **not** localize.
- String(localized: "Pusti") — explicit, for strings built outside a view.

Which is why the view model says:

```

var playButtonLabel: String {
    if isPlaying { return String(localized: "Pauziraj") }
    return offersReplay ? String(localized: "Pusti od početka") : String(localized: "Pusti")
}

```

A String handed to Text silently skips the catalog, and the bug is invisible in the source language — which is the whole reason it survives to release. Text(verbatim:) is the other half: it says “this is deliberately not a key”, which stops the extractor from collecting a stream URL or a bank account number as a translatable string.

When the same word needs two translations

```

// Its own key: "Ostalo" is also the last tab, which is "More" in
// English, where this one is "Other".
case .ostalo: return String(localized: "show.other", defaultValue: "Ostalo")

```

Two places in the app say **Ostalo** in Serbian and need different English words — the tab bar’s overflow is *More*, the show bucket is *Other*. Sharing a key would force one to be wrong. The fix is a **symbolic key with a default value**: show.other is what the catalog is keyed by, Ostalo is what ships if no translation exists.

This is the honest answer to a real design question — *should keys be symbolic or should they be the source string?* — and the position this codebase takes is worth defending in an interview:

Source strings as keys, because they make the code readable and mean the source language never needs a translation pass. Symbolic keys only where identical source strings need to diverge, or where the string alone is too ambiguous to translate. Going fully symbolic would mean 147 opaque identifiers in the code and a Serbian translation file for a Serbian app.

Plurals, which Serbian makes non-negotiable

English has two plural forms. Serbian has three: *one*, *few* (2–4), *other* (5+). 1 epizoda, 3 epizode, 7 epizoda. String Catalogs handle this with plural variations per language, and the code stays a single interpolation:

```
let episodes = String(localized: "\(downloadedCount) epizoda")
```

This is the single best justification for String Catalogs over hand-written `.strings` files, and a good thing to be able to state: the grammar rules are CLDR's, per language, and you do not write branching code for them.

In an interview

Question: “How do you approach localization?”

Answer: “String Catalogs, source language Serbian, and the two things people get wrong. First, `Text(aString)` does not localize — only `LocalizedStringKey` does — so anything built outside a view has to use `String(localized:)`, and the bug is invisible in the source language. Second, iOS picks the app language from the device's preferred-language list, not from the region, so a Serbian app on a phone with English listed comes up in English. We set `AppleLanguages` in our own defaults domain on first launch — the same key the system's per-app Language setting writes — so it is a default rather than a lock, and the user's own choice afterwards is never touched.”

The bonus story: the bundle-swizzle that did nothing, disproved with a log line, and replaced by one `.environment(\.locale,)`.

12. Analytics that measure nobody

The problem

One real question: **does anyone use the bookmarks feature?** It was the reason the app was rewritten, it cost weeks, and there is no way to know from the outside — bookmarks live in the user's own private CloudKit database, which the developer cannot read and should not be able to.

The wrong instinct is to reach for a general-purpose SDK. That brings a device identifier, IP-based geolocation, a session graph, a privacy manifest with tracking domains, an App Tracking Transparency prompt, and a privacy label that says far more than the question needed.

The code

```
/// Counts, and only counts.
///
/// What a person bookmarks lives in their own iCloud database, which is not
/// readable from here and should not be. So the one question worth asking –
/// does anyone use the bookmarks at all - can only be answered by the app
/// saying so, and it says so without saying anything about the person: no
/// identifier, no note, no episode, no position. An event name and, at most,
/// which button it came from.
///
/// Aptabase keeps no device identifier, cookie or fingerprint, and its
/// European servers keep no IP address, which is what makes this a count
/// rather than a record about someone.
enum Analytics {

    private static let appKey = "A-EU-9320487015"

    /// Debug builds are the developer's own use, and would drown a few
    /// thousand real listeners in one afternoon of testing.
    private static var isEnabled: Bool {
        #if DEBUG
            return false
        #else
            return !appKey.isEmpty && !AppSettings.shared.suppressesUsageStatistics
        #endif
    }
}
```

Four events, and the comments say what each is for:

```
enum Event: String {
    /// A bookmark was made, and from where.
    case bookmarkCreated = "bookmark_created"
    /// The list of them was opened.
    case bookmarksOpened = "bookmarks_opened"
    /// One of them was tapped and led back into the audio. The one that
    /// matters: making bookmarks is easy, returning to them is the test.
    case bookmarkPlayed = "bookmark_played"
    /// How many a person has, as a range, once a week at most. Says how
    /// many people have any at all without ever describing one.
    case bookmarksHeld = "bookmarks_held"
}
```

bookmark_played is the designed one. Creation counts measure curiosity; return counts measure value. Picking the metric that can disconfirm your own feature is a product instinct, and interviewers notice it.

Bucketing before it leaves the device

```

/// A range rather than a number, sent at most once a week: enough to tell
/// nobody from a few from a hundred, not enough to follow anybody.
static func recordBookmarksHeld(_ count: Int, defaults: UserDefaults = .standard) {
    guard isEnabled else { return }

    let last = defaults.object(forKey: countedKey) as? Date ?? .distantPast
    guard Date().timeIntervalSince(last) > 7 * 24 * 60 * 60 else { return }
    defaults.set(Date(), forKey: countedKey)

    record(.bookmarksHeld, ["range": range(of: count)])
}

static func range(of count: Int) -> String {
    switch count {
    case 0: return "0"
    case 1...5: return "1-5"
    case 6...20: return "6-20"
    default: return "20+"
    }
}

```

Two independent privacy mechanisms, and being able to name both is the substance of this chapter:

- **Generalisation.** 47 is close to a fingerprint when combined with anything else; 20+ is not. The bucket is applied **on the device, before transmission** — so the precise number never exists off the phone, which is a much stronger guarantee than a promise to discard it at the server.
- **Rate limiting.** Once a week. A count that can be sent on every launch becomes a usage pattern: send 6-20 every morning at 07:40 and you have described someone's commute. Frequency is data even when the payload is not.

The switch

```
return !appKey.isEmpty && !AppSettings.shared.suppressesUsageStatistics
```

The user-facing control is phrased **negatively and defaults to off**: “*Ne šalji anonimnu statistiku*” — “Do not send anonymous statistics”, unchecked. That is a deliberate choice worth defending. A switch labelled positively and defaulting to on is technically the same behaviour and reads as a pre-ticked consent box. Phrasing the control as the *opt-out it actually is* means the default state of the UI matches the default state of the world: nothing is switched on that the user did not switch on.

And `appKey.isEmpty` disables everything, which is the correct behaviour for a fork of a public repository. This one is going on GitHub as a reference project; a clone should not silently report to the original author's dashboard. The key is committed, though, so that is a default rather than a guarantee — a fork has to blank it, and that belongs in the README rather than in anybody's assumptions.

The dependency, made optional at compile time

```

#if canImport(Aptabase)
import Aptabase
#endif

static func start() {
    #if canImport(Aptabase)
    guard isEnabled else { return }
    Aptabase.shared.initialize(appKey: appKey)
    #endif
}

```

`canImport` rather than a protocol wrapper: the code **compiles wherever the module is not available to the target**, and analytics then does not exist in that build rather than existing and being disabled. For a third-party SDK on a dependency-light project that is the right amount of abstraction; a protocol and a null implementation would be more ceremony for the same outcome.

State the limit precisely, because it is easy to overclaim: `canImport` is a compile-time question about the module. It does **not** remove the package from the project — it is in `project.pbxproj` — and it does not promise that a fresh clone builds offline without resolving it. What it buys is that the call sites do not care, and that a target built without the package still compiles.

What may be collected without consent — the actual legal shape

Worth being able to state, because most engineers get the boundary wrong in both directions.

App Tracking Transparency is narrower than people think. It is required for *tracking*: linking user or device data to data from **other companies'** apps or websites for advertising or data-broker purposes. First-party analytics that never leave your own measurement, and carry no advertising identifier, are not tracking and do not require the ATT prompt.

GDPR is the binding constraint, and it turns on whether the data is personal. No device identifier, no IP retention, no cookie, no fingerprint, a bucketed count — that is aggregate measurement rather than personal data. EU hosting removes the transfer question entirely.

The App Store privacy label must still declare it: *Usage Data → Product Interaction*, **not linked to the user, not used for tracking**. Collecting nothing personal does not mean declaring nothing — and the label covers the app *as shipped*, SDKs included.

The caveat that keeps all of the above honest. Everything in this section is a claim about what the app sends. The vendor also sends things the app did not write — a session identifier, a timestamp, an OS and locale string — and “I did not attach an identifier” is not the same as “no identifier is attached”. Before repeating any of this in an interview, say that you checked the SDK’s own payload and retention, because the difference between an engineer who knows the boundary and one who has read a marketing page is exactly that sentence.

The honest ordering, and the thing to say: “The switch is there because the label and the law are not the whole of it. People should be able to say no to being counted even when being counted is anonymous, and the cost of offering that is one row in Settings.”

In an interview

Question: “How would you add analytics to an app?”

Answer: “Start from the question, not the SDK. We had exactly one — does anyone use the bookmarks — so the implementation is four event names and a bucketed weekly count, with the bucketing done on the device so the precise number never leaves it. The vendor was chosen for what it does not collect: no device identifier, no IP retention, EU hosting, which keeps it aggregate measurement rather than personal data. ATT is not required because we are not tracking across other companies’ properties, but the privacy label still declares *Usage Data → Product Interaction*, not linked, not for tracking. And there is an opt-out phrased as an opt-out, defaulting to off, because people should be able to decline being counted even when the counting is anonymous.”

Follow-up to expect: “What if you exceeded the free tier?” — a real question that was asked during this work. The answer is the interesting part: the event volume here is bounded by *bookmark interactions*, not by sessions or screen views, which is the difference between thousands of events a month and millions. **Choosing a metric that is cheap to collect is part of choosing the metric.**

Part IV – Quality

13. Testing beyond the simulator

The problem

Three of the four things this app does badly wrong resist an ordinary unit test, each for a different reason — and it is worth being exact about which, because “you can’t test that” is usually laziness wearing a disguise:

- **A car.** Xcode does ship a CarPlay simulator, and it covers what it is good for: templates, navigation, the scene lifecycle. What it does not cover is the part that broke — audio routing, the cable being pulled, the phone locked, the app launched by the car with no window. That needed hardware, and then it needed a test that did not involve a car at all.
- **A second device.** CloudKit sync needs two signed-in machines, a network and minutes of patience.
- **Real audio.** AVPlayer is asynchronous in five places (chapter 7) and none of them are deterministic.

The instinct is to conclude that none of it is testable. The answer is to be precise about *what* is untestable, and then test the rest — which turns out to be nearly all of the logic.

The seams

Three protocols, and each one exists for a specific test that was otherwise impossible.

```
protocol PlaybackEngineType: AnyObject { ... } // an engine with no AVPlayer

protocol EpisodeLookup: AnyObject { // a store with no database
    func podcast(with id: UUID) -> Podcast?
    func refreshFromStore()
    func lastListened() -> Podcast?
}

protocol ProgressRecording: AnyObject { // a writer that writes nowhere
    func recordProgress(position: TimeInterval, hasFinished: Bool, for id: UUID)
}
```

EpisodeLookup has default implementations, which is a small but real design decision:

```
extension EpisodeLookup {
    func refreshFromStore() {}
    func lastListened() -> Podcast? { nil }
}
```

A test double implements the one method it cares about. Without the defaults, every fake has to implement every method and each new protocol requirement breaks every existing test — which is the friction that makes people stop writing them.

The fake engine

```

/// An engine with no player behind it. It reports what it is told to report,
/// and writes down what it was asked to do.
private final class FakePlaybackEngine: PlaybackEngineType {

    private let sourceSubject = CurrentValueSubject<PlaybackSource?, Never>(nil)
    private let timeSubject   = CurrentValueSubject<TimeInterval, Never>(0)
    private let movedSubject  = PassthroughSubject<TimeInterval, Never>()

    private(set) var playCalls: [(source: PlaybackSource, position: TimeInterval?)] = []
    /// Flattened, so a test can ask "was it started from nowhere in
/// particular" without going through two layers of optional.
    var lastPlayPosition: TimeInterval? { playCalls.last?.position ?? nil }
    private(set) var seekCalls: [TimeInterval] = []

    func play(_ source: PlaybackSource, startingAt position: TimeInterval?) {
        playCalls.append((source, position))
        sourceSubject.send(source)
        timeSubject.send(position ?? 0)
        playingSubject.send(true)
    }

    // Driving it from a test
    func advance(to time: TimeInterval) { timeSubject.send(time) }
    func reportDuration(_ duration: TimeInterval) { durationSubject.send(duration) }
    func stopPlaying() { playingSubject.send(false) }
}

```

It is both a **stub** (it answers) and a **spy** (it records), and the three methods at the bottom make it a **driver** — a test can move the playhead, report a duration, and stop playback, in that order, deterministically, in microseconds. The mirror of `CurrentValueSubject` for state and `PassthroughSubject` for events is not decoration: it reproduces the real engine’s semantics, where a new subscriber must immediately learn the current position but must *not* be told about a seek that happened before it existed.

`lastPlayPosition` deserves its comment. `playCalls.last?.position` is `TimeInterval??`, and the difference between “no play call” and “played from nowhere in particular” matters — flattening it once in the fake keeps every assertion readable.

The car, as a test

The bug from chapter 6, with the user’s real numbers:

```

/// The car: resumed at 57:45, a quarter of an hour of driving, the cable
/// pulled. iOS pauses the audio, and that pause is what gets written.
func testAListenInTheCarIsWrittenDownWhenTheCableIsPulled() {
    engine.play(.podcast(alarm), startingAt: 3_465)
    engine.advance(to: 4_365)
    engine.stopPlaying()

    XCTAssertEqual(progress.calls.last?.id, alarm.id)
    XCTAssertEqual(progress.calls.last?.position ?? 0, 4_365, accuracy: 1)
    XCTAssertEqual(PlaybackStateStore(defaults: defaults).saved?.position ?? 0, 4_365, accuracy: 1)
}

```

And the test class says why it looks like this:

```

/// Everything about when a listen is written down, against the engine alone –
/// no player view model, no screen. That is the point: the car starts the app
/// with no phone window, and the listen still has to be written.

```

The test’s shape encodes the bug’s cause. A test that constructed a `PlayerViewModel` would pass and prove nothing, because the view model is exactly what CarPlay does not create. Being able to say “the absence of a view model in this test is the assertion” is a strong thing to have ready.

The second device, as a test

Two approaches, for two different questions.

In-memory, with rows written straight into the store — for merge rules:

```
override func setUp() {
    database = AppDatabase(inMemory: true)
    repository = PodcastRepository(database: database)
    episode = makeEpisode(title: "Alarm")
    repository.save(episode)
}

/// A row as another device would have left it: written straight into the
/// store, not through the repository.
private func insertRow(position: Double, at date: Date?, ...) {
    let row = EpisodeStateEntity(podcastId: episode.id, title: episode.title)
    ...
    database.context.insert(row)
    XCTAssertNoThrow(try database.context.save())
}
```

Two real containers on the same files — for the question of whether a second writer's change is visible:

```
/// `inMemory` is for tests, which want a store that starts empty and
/// leaves nothing behind. `storeDirectory` is for the tests that need the
/// store to be a file two containers can open at once - which is what an
/// import from iCloud looks like from inside the app: something else
/// writing to the same store. Either one turns syncing off: a test has no
/// business reaching iCloud.
init(inMemory: Bool = false, storeDirectory: URL? = nil) {

func testAReadAfterAnotherWriterSeesTheNewValue() {
    let phone = PodcastRepository(database: AppDatabase(storeDirectory: directory))
    phone.save(episode)
    phone.recordProgress(position: 600, hasFinished: false, for: episode.id)
    XCTAssertEqual(phone.podcast(with: episode.id)?.playedPosition, 600)

    let otherDevice = PodcastRepository(database: AppDatabase(storeDirectory: directory))
    otherDevice.recordProgress(position: 5_400, hasFinished: false, for: episode.id)

    XCTAssertEqual(phone.podcast(with: episode.id)?.playedPosition, 5_400)
    XCTAssertEqual(phone.resumePosition(for: episode.id) ?? 0, 5_397, accuracy: 0.5)
}
```

That is genuinely two independent ModelContainers on one store file — as close to CloudKit's own behaviour as you get without CloudKit.

Testing what iCloud will accept

This one is unusual and worth stealing. CloudKit rejects a bad schema *at store-open time*, the app catches it and opens unsynced, and **nothing on screen says so** — so a model change that breaks syncing ships silently and looks fine on the developer's machine, which already has a working store.

```

/// iCloud refuses a model with a unique constraint, or with a required value
/// that has no default, and it refuses it when the store is opened. In the
/// app that refusal is caught and the store opens again without syncing: it
/// keeps working, and it never syncs again, and nothing on screen says so.
/// This opens the synced half exactly as the app does and fails where the
/// app would go quiet.
func testTheSyncedModelsAreAcceptableToICloud() throws {
    let configuration = AppDatabase.syncedConfiguration(at: directory.appendingPathComponent("Synced.store"))

    XCTAssertNoThrow(try ModelContainer(for: Schema(AppDatabase.syncedModels),
                                       configurations: configuration))
}

```

A test that turns a silent runtime degradation into a red build. That is the general pattern: wherever production code recovers from a failure by quietly doing less, a test should assert the failure does not happen.

Saying what you are not testing

SyncStateTests.swift opens with its own scope statement:

```

// iCloud itself is not here. Whether a row leaves one device and reaches
// another is Apple's machinery, over the network, on its own schedule; a test
// of that needs two signed-in devices and minutes of waiting, and is a
// different kind of test from these.

```

Steal this habit. It prevents the two failure modes of a test suite: false confidence (“sync is tested”) and false despair (“sync is untestable”). The boundary is: **their machinery is theirs; our rules are ours; test ours.**

Where the fake stops being enough

One test drives a real AVPlayer:

```

final class PlaybackReplayTests: XCTestCase {
    func testActualPlayerCanResumeAndExplicitlyReplayAfterEOF() throws

```

Because end-of-file behaviour is a property of AVPlayer itself: what currentTime reports after AVPlayerItem-DidPlayToEndTime, and whether play() on a finished item does anything. A fake would answer whatever it was written to answer, which is a tautology. **Know which of your assumptions are about your code and which are about the framework, and test the second kind against the framework.**

The async helper

```

/// The view model takes everything from the engine on the main queue, a
/// runloop later. This lets those deliveries land before the assertions.
private func flush() {
    let delivered = expectation(description: "main queue drained")
    DispatchQueue.main.async { delivered.fulfill() }
    wait(for: [delivered], timeout: 1)
}

```

A fence rather than a sleep. Enqueue a block behind everything already queued; when it runs, everything before it has run. Deterministic, and it takes microseconds instead of a fixed delay. XCTest has no built-in for this and most codebases use sleep(0.1), which is both slower and flaky.

Know its limit, because it is the kind of helper people over-trust: it waits only for work *already* on the main queue. Anything that hops elsewhere and comes back later is not covered, and a test for that has to observe the completion it actually cares about — which is what the real-AVPlayer test below does with expectations.

The test that failed honestly

Worth ending on, because it is the best small story in the suite. A test started failing with 600 where 597 was expected. The commit message diagnoses it:

The failing test was the honest kind: it handed the player a row dated now without the slot that the recorder writes beside it, so the player correctly read it as a listen from elsewhere. It writes both now.

The production code was right. The **fixture** was impossible — it constructed a state no real device could be in, because the recorder always writes both records together. The test was fixed, not the code.

Being able to tell that story is worth a great deal, because the default reaction to a red test is to change the code until it is green, and *“the fixture described a state that cannot exist”* is a diagnosis most people never reach.

In an interview

Question: *“How do you test something that depends on hardware you do not have?”*

Answer: “Separate what is yours from what is the platform’s. Everything about *when* a listen is written down is ours, so it is tested against a fake engine with no `AVPlayer` in it — and pointedly with no view model either, because the bug it was written for was that CarPlay never creates one. Everything about how `AVPlayer` behaves at end of file is theirs, so there is one test that drives a real player over a generated file. For sync, the merge rules are ours and are tested in an in-memory store with rows written straight in, the way an import would; whether a row crosses the network is Apple’s and is not tested at all, and the test file says so at the top.”

Best follow-up to invite: *“Is there anything you test that most people do not?”* → the schema-acceptance test. A model change that breaks CloudKit is caught and swallowed by the app’s own fallback, so without that test it ships silently and everyone’s sync stops.

14. Reading a code review like a senior

Scope note for anyone reading this as repository documentation rather than as interview prep: the app is on SwiftData and has no Realm dependency. The migration argument below is a record of a decision that was taken, not an open question, and the legacy bookmark categories mentioned in passing were never released — the reader that once handled them has since been removed for that reason.

Why this chapter exists

There is an interview question that separates people cleanly: “tell me about a time you got difficult code review feedback.” Most answers are about diplomacy. The better answer is about **triage** — how you decide which findings are real, which are opinions, and which are wrong.

This codebase went through three rounds of automated review on one pull request (refactor/swiftdata → development). The outcome was five genuine defects fixed, several smaller ones, one finding argued down, and one piece of feedback that turned out to be *more right than the reviewer knew*.

The five that were real

From the commit that answered the first round — af0a2fe, “Answer the review: four defects, and the reading that wrote”:

1. Infinite recursion after an interruption.

Play after a failed item recursed until the stack ran out. `resume()` saw the failure and called `play()`; `play()` saw the same source with a player still attached and called `resume()`. A failed item is no longer “already loaded”, so it is torn down and built again, at the second it stopped on.

A genuine crash on a path — another app taking the audio session — that is hard to reach by hand and trivial to reach in real use. (Chapter 7, bug 4.)

2. An adopted position written down as a listen.

A position adopted from another device was indistinguishable from listening: the phone was moved to it while paused, and the next write stamped it with this moment — handing this device the account’s newest listen over the device that had actually listened. The recorder is now told when a move was not a listen. Dragging the scrubber still counts, because that is somebody here.

The most valuable finding of the three rounds, and the one that required the most thought to fix properly. (Chapter 6.)

3. Two read paths, one rule.

A list and a single episode read the same rows and gave different answers... An episode favoured on one device and listened to later on another was missing from the favourites list until its player had been opened.

(Chapter 5.)

4. A read that wrote.

Reading an episode folded its duplicate rows away and deleted them — a store write in the middle of a read. Folding is the next write’s job; reading answers from the rows as they are.

5. A button that promised something it could not do.

A bookmark whose episode this device has never fetched showed a play triangle and did nothing when pressed. Bookmarks sync and the episode cache does not, so it now says what it is until the episode turns up.

This one is a *consequence of the architecture in chapter 4* that nobody had followed through: bookmarks sync, the episode cache does not, therefore a bookmark can arrive on a device that has never heard of its episode. The review found the symptom; the fix required understanding the design.

Plus the smaller ones, all real:

the notification observers are held and given back in `deinit`; “carry on with” asks the store for the

ten rows it needs instead of every row ever written; and an unchanged duration no longer redraws every screen watching the player.

The one that was argued down

The review recommended importing the old Realm database on upgrade, so users would not lose favourites and bookmarks. It is a reasonable default position and it was rejected — with the reasoning written into the code rather than into a comment thread that disappears when the PR is merged:

```
/// Nothing is carried over from Realm, which this app used until 3.0, and  
/// that is a decision rather than an oversight.  
///  
/// An import was written once and thrown away: the old rows were keyed by  
/// identifiers that mean nothing here, so every episode had to be matched  
/// by its media URL, and a mismatch wrote a listening position onto the  
/// wrong episode - worse than the empty start it was meant to avoid. What  
/// is lost is favourites, bookmarks and the record of which files were  
/// downloaded; the catalogue comes back from the feed on the first  
/// refresh, and the downloaded files are still on disk and are found  
/// again when their episodes are downloaded once more.  
///  
/// It belongs in the release notes for the version that ships this.
```

Four things make this a *senior* disagreement rather than a stubborn one:

- **It engages with the recommendation.** The import was written, not dismissed.
- **It names the failure mode concretely.** Matching by URL is fuzzy, and a mismatch writes a position onto the wrong episode — silently corrupt data, which is worse than an empty start.
- **It states the cost honestly.** Favourites, bookmarks, download records. No minimising.
- **It carries an obligation.** *“It belongs in the release notes.”* A decision to lose user data is only defensible if the user is told.

The general principle, and the sentence to use: **a rejected review finding should leave a written trace in the code, not in the pull request.** The PR is read once; the file is read forever, and the next person to notice the missing import deserves the reasoning without archaeology.

The one that was more right than the reviewer knew

The best moment of the three rounds. A test was meant to prove that re-reading the store before a decision is what makes another device's position visible. The review poked at it. The investigation found something worse — db64502, *“Stop claiming the re-read was the fix”*:

The test meant to prove that re-reading the store before a decision is what makes a position from another device visible **passed just the same with the re-read taken out**. The repository keeps no rows between reads: it fetches every time and hands out copies, so there is nothing stale to refresh. The “reading a photograph taken at launch” explanation was a theory that was never proven, and what was actually wrong at the time was upstream — the other device's row had not arrived.

So the test was not testing what its name claimed, the explanatory comment was fiction, and a real bug had been “fixed” by a change that did nothing while the actual cause resolved itself elsewhere. Three failures in one.

The resolution is the interesting part, because the code did **not** get deleted:

The re-read stays, because it is free and it becomes necessary the day something holds on to rows. Its comments now say that, and the test is now a test of what is true: a read after another writer sees the new value, and will stop doing so if anyone starts caching rows.

And the comment in the source now says so plainly:

```

/// A precaution, not a fix. It was written on the theory that a context
/// kept for the life of the app goes on answering with rows it read before
/// an import changed them... StoreChangeTests says otherwise: the
/// repository hands out copies and fetches on every read, so nothing it
/// read earlier is still held... What was actually wrong then was upstream
/// - the other device's row had not arrived.
///
/// It stays because it costs nothing - a context holds no data until
/// something is fetched through it - and because it keeps that true if
/// something ever does start holding on to rows.

```

Three separate senior behaviours in one commit, and this is the story to tell if you are asked about intellectual honesty:

- **The claim was tested by removing the code.** If a test passes with the fix removed, the test does not test the fix. That is a five-second experiment almost nobody runs.
- **The fiction was corrected rather than deleted.** A wrong comment is worse than no comment, because it is trusted.
- **The code stayed for a different, smaller, true reason.** “It is free and it becomes load-bearing the day someone adds caching” is a real justification. Keeping it under the old false one would not have been.

The finding that generalised

Round two produced a fix, 50d2c5d, that is worth quoting because the *reviewer's finding was narrower than the bug*:

The flag that keeps another device's position from being written down as a listen was cleared by exactly one thing: a drag of the scrubber in our own player. The skip buttons go straight to the engine, and so do the lock screen and the car, so fifteen seconds forward on a paused episode that had just taken a position from the Mac was not saved at all — the recorder still thought the position was somebody else's.

The engine is the one place all of them pass through, so it is the one place that can say whose move it was.

The review found one missing case. The fix did not add that case — it moved the responsibility to the only layer where the question is answerable at all, which closed every case including the ones nobody had listed.

That is the difference between fixing a finding and fixing its cause, and it is exactly what an interviewer is listening for. The weak version of this answer is “I addressed all the comments.” The strong version is “one of the comments was a symptom of a boundary being in the wrong place, and moving it closed four bugs including two nobody had found yet.”

How to talk about AI-assisted review

Say plainly that the reviews were automated, because the interesting part is not who wrote them:

Three rounds of automated review on one PR. Five real defects — one crash, one data-correctness bug in sync, two consistency bugs, one broken affordance — plus the usual smaller ones about observer lifetimes and over-fetching. One recommendation I argued down and wrote the reasoning into the source rather than the PR thread. And one finding that, when I checked it properly, showed that an earlier fix of mine had never done anything and its explanatory comment was fiction.

What I took from it: an automated reviewer is very good at the class of bug that is local and mechanical — a lifetime, a missing case, a read that writes. It is not good at deciding whether a finding is a symptom. Both of the fixes I am happiest with came from treating a narrow finding as a question about where a responsibility belongs.

In an interview

Question: “Tell me about difficult code review feedback.”

The structure: triage, not diplomacy.

1. **What was real and got fixed** — name the most serious one specifically.
2. **What you disagreed with, and how you recorded the disagreement** — the Realm import, with the reasoning in the code and the obligation to the release notes.
3. **What the review taught you that it was not trying to** — the re-read that never did anything, found by deleting it and watching the test still pass.

The line to have ready: *“The finding was real but narrow. Treating it as a question about where the responsibility belonged, rather than as a missing case to add, is what closed the ones nobody had found.”*

15. Honest weaknesses

Volunteering a real weakness is the highest-leverage thing you can do in a senior interview, and it only works if the weakness is **specific, ranked, and accompanied by the fix you have not made yet**. “I sometimes over-engineer” is not a weakness, it is a humblebrag. What follows is the real list for this codebase, in the order it would actually be addressed.

1. Concurrency is held by convention, not by the compiler

The problem. `SWIFT_VERSION = 5.0` — which selects the Swift 5 language mode, not an old compiler — and strict concurrency checking is not turned on. `AppDatabase.context` is a hand-made `ModelContext`, deliberately non-isolated, and its rule is documented rather than enforced:

```
/// Deliberately not `container.mainContext`, which is @MainActor and would  
/// drag the annotation through the repository and everything that calls it.  
/// A context of our own is nonisolated, and this one is only ever touched  
/// from the main thread anyway - every write reaches it from a network  
/// completion that already hopped there.
```

The KVO callbacks in `PlaybackEngine` hop to main before touching state, and the seek completions now do too — they did not until a review went looking, which is the point. Those are the paths somebody has audited. `ModelContext` is explicitly not thread-safe and is confined to its owner by documentation alone.

The fix, in order: turn on `SWIFT_STRICT_CONCURRENCY = complete` and read the warnings; annotate `PlaybackEngine` and `PodcastRepository` `@MainActor`; work through the remaining callback and synchronous entry points, of which the seek handlers were only the first. The reason it has not been done is honest: it is a refactor with no user-visible benefit, scheduled for after 3.1 ships. What *did* get done during review is the queue hop and invalidating completions at teardown — a fix to two known paths, not a proof about the rest, and worth describing that way.

2. Silent degradation

Three places where the app quietly does less and never says so:

- **Sync off.** This is worse than it first looks. `isEphemeral` marks the *in-memory* tier, so the middle tier — store opened, syncing refused — is not represented by any state at all, let alone shown. A user whose schema was refused, or whose entitlement is missing on a build, gets an app that works perfectly and never syncs again, and nothing anywhere knows.
- **In-memory store.** Third tier of the same fallback. Favourites do not survive the session and nothing says so.
- **Analytics.** Correct to be silent, but it is the same shape.

The fix. A single “iCloud sync is unavailable” row in Settings, driven by `isEphemeral`. Perhaps thirty lines. It is the highest value-per-line item on this list.

3. NetworkManager set timeouts that had no effect — fixed

Kept on the list because the *story* is the useful part, and because it is a good one to volunteer: small, subtle, and everyone has written it. This is what it looked like:

```
init(session: URLSession = URLSession.shared) {  
    self.session = session  
    self.session.configuration.timeoutIntervalForRequest = Constants.timeoutIntervalForRequest  
    self.session.configuration.timeoutIntervalForResource = Constants.timeoutIntervalForResource  
}
```

`URLSession.configuration` is documented as returning **a copy**. These two lines mutate a temporary and are discarded. The app runs on the default 60-second request timeout, not the intended 15.

The fix, now shipped: build the configuration first, make the session from it, and leave an injected session exactly as it arrived.

```

init(session: URLSession? = nil) {
    if let session {
        self.session = session
    } else {
        // URLSession.configuration returns a copy; configure before creation.
        let configuration = URLSessionConfiguration.default
        configuration.timeoutIntervalForRequest = Constants.timeoutIntervalForRequest
        configuration.timeoutIntervalForResource = Constants.timeoutIntervalForResource
        self.session = URLSession(configuration: configuration)
    }
}

```

Mutating a shared session's configuration would have been wrong anyway — `URLSession.shared` is used by other things, which is the second reason the old signature was a trap.

The tests are the part worth copying. They assert against `manager.session.configuration`, not against a local configuration object — checking the object you just built would have passed against the broken version too, which is how a bug like this survives a test suite.

4. The network layer is completion-handler based

`NetworkManaging` is four overloads of completion: `@escaping (Result<...>) -> Void`, and the download path carries a separate `progressHandler`. It predates the rest of the refactor. The consequences are ordinary and real: nesting at call sites, no cancellation, no structured concurrency, and error propagation by convention.

The fix. `async` throws for the request paths and `AsyncStream<Double>` or a `URLSession` delegate for download progress. Deliberately *after* the concurrency migration in item 1, because doing it first would create `@MainActor` questions that the annotations should answer.

5. Un-favouriting cannot sync

Covered in chapter 5. The flag union converges but is grow-only, so clearing a favourite on one device is re-asserted by any device still holding true.

The fix. Store the decision's timestamp — `favoritedAt: Date?` — and merge it by the same last-write-wins rule the position uses. **Why it has not happened:** it is a schema change on a *synced* store, which is the one migration you cannot roll back, and it was not worth taking before 3.1 shipped.

6. Last-write-wins assumes comparable clocks

`playedAt` is `Date()` on each device. Two devices with meaningfully skewed clocks would resolve in the wrong order. Across one person's own Apple devices this is a theoretical problem, not a practical one — but it is an assumption, and it should be stated as one rather than discovered.

There is a smaller, sharper version of the same gap: `isNewer` compares with `>`, so two rows written in the same millisecond are “not newer” in both directions and the winner is whatever order the fetch returned. No total order, no convergence guarantee — even with perfect clocks.

The fix, in increasing order of cost: break the tie on a stable writer id, which is cheap and removes the non-determinism; then, if it ever mattered, a Lamport counter or vector clocks, which detect concurrent edits but still cannot tell you the real-time order of two disconnected actions. Almost certainly not worth the second one here, and saying “*not worth it, and here is what it would cost*” is a better answer than either building it or not knowing.

7. `Insecure.MD5` invites a conversation it does not need

Chapter 3 defends it correctly: this is identity, not security. But SHA256 truncated to 16 bytes is the same code, the same determinism, and removes the conversation — and the name `Insecure` in a source file is a liability in any security review.

The cost of the fix: every derived id changes, so every stored row is orphaned. It is a one-time data migration for a cosmetic gain, which is why it has not been done — and that is a complete answer.

8. DEBUG instrumentation still to be stripped

The `#if DEBUG` logging is extensive — CloudKit events, store row dumps, buffer state, CarPlay template results, the language switch, the erase- everything control, log export. It is genuinely useful and it is all compiled out of Release. What remains is the discipline of confirming that before each submission, which is currently a checklist rather than a build-phase check.

9. Test coverage is deep, not wide

Ninety-six tests in about 1 700 lines, concentrated where the bugs were: listening rules, sync merge, player state, the recorder, stream metadata, and now the session configuration. Almost nothing covers the rest of the network layer, the download manager, the bookmark library's live-matching path, or any view. And the default template tests are still sitting in `AlarmDMTests`, beside the two real ones that were added to it:

```
final class AlarmDMTests: XCTestCase {
    func testExample() throws { ... }
    func testPerformanceExample() throws { ... }
}
```

The honest framing: the coverage follows the risk, which is the right instinct, but “we never had a bug there” is not the same as “there is no bug there.” `LiveBookmarkMatcher` in particular is pure, deterministic, arithmetic over dates — the cheapest possible thing to test, and untested.

How to use this list

Pick **one** and volunteer it before being asked. The best candidates:

- **Item 1** if the role is modern Swift — it shows you know what Swift 6 actually changes, and that you can tell “correct” from “proven correct.”
- **Item 3** if you want to demonstrate that you read your own code critically — it is small, real, and the kind of thing that only gets found by someone looking properly.
- **Item 5** if the conversation is about distributed state — it shows you understand *why* your own merge rule is the shape it is, including where it breaks.

And the framing that makes any of them land:

“The thing I would fix first is X. I know what the fix is — Y. The reason it is not done is Z, and Z is a scheduling decision, not a blind spot.”

Part V – Drills

16. Question bank

Answers in note form. The point is not to memorise the wording; it is that every answer here has a real file behind it, so a follow-up question lands on something you have actually done. Chapter references point at the detail.

Swift language

Q. Value types versus reference types – how do you decide?

Podcast is a struct: it is data, it is copied freely between the repository, the engine and three view models, and every domain rule on it is a pure function of its stored values (ch. 9). PlaybackEngine is a class: there is exactly one AVPlayer in the process and everything must see the same one (ch. 2). The decision rule: *does sharing mutations matter?* If yes, reference. If no, value – and then the rules on it become trivially testable.

Q. When does Equatable bite you?

The best story in the codebase (ch. 2). Podcast carries both what an episode *is* and what you *did with it*. The moment progress was written, the refreshed Podcast was no longer `==` to the one the engine held – same audio file. Code asking “is this already playing?” with `==` got “no”, reloaded, and restarted from zero. Hence `contentId` and `isSameContent(as:)`: **identity is about the audio, equality is about the whole value**.

Q. Failable initialisers – when?

`LiveTrack(raw:)` (ch. 8). Validation at the boundary means a value that exists is a value worth using; there is no `isValid` for callers to forget.

Q. Why not `hashValue` for a stable key?

Swift’s `Hashable` is seeded per process and is explicitly not stable across launches. `UUID.stable(from:)` hashes deterministically instead (ch. 3).

Q. Defend a try!.

```
container = try! ModelContainer(for: schema, configurations: /* in memory */)
```

A deliberate fail-fast: reaching it means not even an in-memory store could be built, so there is no persistence layer left to fall back to. Do not overclaim the cause – a malformed schema is the likeliest, not the only one. A forced operation is defensible when no recovery remains, and indefensible as a shortcut past an error that depends on the environment (ch. 4).

SwiftUI

Q. @StateObject vs @ObservedObject vs @EnvironmentObject?

`@StateObject` where the object is created and owned – `RootView` makes the `PlayerViewModel`. `@EnvironmentObject` for everything below it, so the mini player, the full-screen player and every row read the same instance. `@ObservedObject` only when it is passed in and owned elsewhere. The failure mode people get bitten by: `@ObservedObject` on something created inline can be rebuilt whenever the owning view value is, taking its state with it – `@StateObject` is what ties the object’s lifetime to the view’s identity instead.

Q. Why does `Text(myString)` not localize?

Text has a `LocalizedStringKey` overload and a `String` overload; a `String` takes the second and skips the catalog. Invisible in the source language, which is why it survives to release. Outside a view, `String(localized:)` (ch. 11).

Q. How do you override the language for one launch?

`.environment(\.locale,)`. SwiftUI resolves a `LocalizedStringKey` against the environment locale, *not* through `Bundle.main.localizedString` — which is why the popular `Bundle` swizzle does nothing for SwiftUI text, proven with a log line either side of the swap (ch. 11).

Q. Adaptive layout without device checks?

Size class for how much room there is (`contentWidth(for:)` caps a reading column at 900 pt), `ProcessInfo` for the interaction model. `ProcessInfo` specifically, because an iOS app on Apple silicon is not `macCatalyst` at compile time (ch. 10).

Concurrency

Q. Where does this codebase enforce the main thread?

By convention, and that is the honest answer. KVO callbacks and seek completions hop through `DispatchQueue.main.async` before touching published state; store writes arrive from network completions that have already hopped. Those are the audited paths, not a proof about the others. Strict concurrency plus explicit `@MainActor` ownership is what would make it checkable (ch. 15).

Q. Why is `ListeningRecorder` subscribed without `.receive(on:)`?

Deliberate. The engine already publishes on main, and a write must land before the caller's next statement: closing the player writes the final position and then clears the slot to reopen. A hop would deliver the write after the clear and resurrect it. *Ordering between a publisher and its caller is part of the contract* (ch. 6).

Q. `CurrentValueSubject` VS `PassthroughSubject`?

State vs events. A new subscriber must immediately learn the current position (`CurrentValueSubject`) but must not be told about a seek that happened before it existed (`PassthroughSubject`). The fake engine mirrors the split exactly, so tests reproduce real subscription semantics (ch. 13).

Q. What is debounce for here?

`NSPersistentStoreRemoteChange` fires dozens of times while `CloudKit` sets up — measured at forty in five seconds on a fresh install. Screens need to know *that* something arrived, not how many times. 400 ms (ch. 4).

AVFoundation

Q. Why do seeks get dropped?

Issued before the item is `readyToPlay`. Park it in `pendingSeek` and replay it from the status observer — **and do not call `play()` in the meantime**, or the file's opening second reaches the speaker before the seek lands (ch. 7).

Q. Why use seek tolerance?

Careful with the API: the convenience `seek(to:)` does *not* default to zero tolerance — sample accuracy is what you request explicitly by passing `.zero` for both. With an exact target the player must have the surrounding data, so backwards is instant and forwards stalls; one second either way is nothing in a three-hour show. Readiness and obsolete callbacks cause stalls too, and were separate fixes (ch. 7).

Q. What must be cleaned up before releasing an `AVPlayer`?

Periodic time observer removed (a documented crash, not a leak), `NSKeyValueObservation.invalidate()`, block-based notification observers removed by token, `AVPlayerItemMetadataOutput` removed from the item, and any in-flight seek flag reset (ch. 7).

Q. Handling interruptions?

`.began` → pause. `.ended` → resume **only if** the options contain `.shouldResume`. Resuming unconditionally makes your app talk over whatever interrupted it. And know that an interruption can leave the item `.failed`, which is not resumable and must be rebuilt (ch. 7).

Q. Lock screen and CarPlay?

`MPRemoteCommandCenter` targets for play/pause/skip/changePlaybackPosition, and `MPNowPlayingInfoCenter` for

the metadata. Two details worth knowing: `MPNowPlayingInfoPropertyIsLiveStream` hides the scrubber for live audio, and `MPMediaItemPropertyPlaybackDuration` is omitted for live rather than set to zero. Commands are enabled/disabled by whether the source is seekable, so live radio shows no skip buttons (ch. 8, 10).

Data and sync

Q. Design offline-first sync with no backend.

Ch. 5, in full. The compressed version: CloudKit private database via `NSPersistentCloudKitContainer`; no unique constraints, so you merge duplicates yourself; positions resolve last-write-wins on a timestamp you write, because the transport's ordering is arrival order and arrival order is not event order; flags union, because a union converges regardless of delivery order. The honest footnote: the comparator has no tie-break, so equal timestamps have no defined winner (ch. 15).

Q. What does CloudKit refuse?

Unique constraints. Non-optional attributes without defaults. Non-optional relationships. Relationships that cross store configurations — which is why `EpisodeStateEntity` links by `podcastId` and denormalises the episode title (ch. 4).

Q. How do you split what syncs from what does not?

By reproducibility. The feed cache comes back from the API and the download path is a fact about one machine; favourites, positions and bookmarks exist nowhere else. Two `ModelConfigurations` in one container; type decides the store (ch. 4).

Q. `ModelContext` — main context or your own?

Ch. 4, including the trade: a hand-made context is non-isolated and does not autosave, which buys explicit transactions with rollback and costs the compiler's help.

Architecture

Q. When is a singleton acceptable?

When the thing is genuinely unique in the process *and* the process has entry points you do not construct. iOS instantiates `CarPlaySceneDelegate` from a class name in the `Info.plist` — you cannot inject into it. So: singleton for the engine, protocol for every consumer, and nothing the singleton owns that a test needs to control (ch. 2).

Q. Where do you put business logic?

On the value type when it is a pure function of stored data (ch. 9); in the repository when it must be identical for every caller, *especially* callers that do not go through your UI (ch. 10). The resume rule lived in the view model once, and the car never went through the view model.

Q. How do you decide what a “rule” is?

If a second entry point would need it and could get it wrong, it is a rule and it belongs one layer down. **A rule that lives on a caller is not a rule, it is a habit.**

Testing

Q. How do you test hardware you do not have?

Ch. 13. Separate your logic from the platform's behaviour; fake the platform, test the logic; write the one real-`AVPlayer` test for the assumption that is actually *about* the framework.

Q. Is there a test most people would not write?

The CloudKit schema-acceptance test. A model that breaks CloudKit is refused at store-open, caught by the app's own fallback, and the app then works perfectly and never syncs — so without the test it ships silently (ch. 13).

Q. A test failed. Was the code wrong?

Not always, and there is a real example: a fixture handed the player an episode row dated now, *without* the slot the recorder always writes beside it. No real device can be in that state, so the player correctly read it as another device's listen. The fixture was wrong (ch. 13).

Privacy and release

Q. What can you collect without consent?

ATT is about tracking across *other companies'* apps and sites; first-party anonymous counts are not that. GDPR is the binding constraint and turns on whether the data is personal — no identifier, no IP retention, bucketed counts, EU hosting. The privacy label still declares *Usage Data* → *Product Interaction*, not linked, not for tracking, and it covers the SDKs as shipped, not just your own code. And there is still an opt-out, because people should be able to decline being counted. Say one more sentence than most candidates do: that the SDK sends fields you did not write — session id, timestamp, OS, locale — and that you verified them rather than assuming (ch. 12).

Q. You are deliberately losing user data on upgrade. Defend it.

Ch. 14. The import was written and thrown away: old rows were keyed by identifiers that mean nothing in the new schema, so matching was by media URL, and a mismatch wrote a listening position onto the wrong episode. Silently corrupt data is worse than an empty start. What is lost is named, what is recoverable is named, and it goes in the release notes.

17. System design: “design a podcast player”

A common take-home or whiteboard prompt, and this app is a complete worked answer. Drive it in this order; each step has a real decision behind it.

Step 1 — Ask what makes it hard, out loud

Do not start drawing. Say:

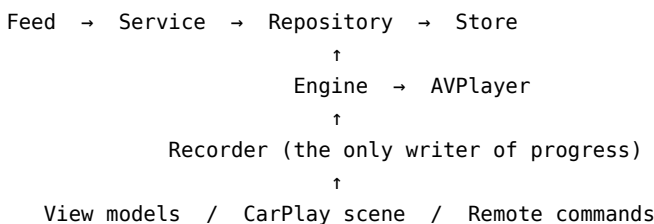
“Playback itself is not the hard part. The hard parts are: where identity comes from if the feed has no ids; where the listening position lives when more than one surface can change it; and what happens when two devices disagree. Can I assume no backend of my own beyond a feed?”

That single sentence reframes the problem from “boxes and arrows” to “state under concurrency”, which is what is actually being assessed.

Step 2 — Identity first

Derive the key from the payload’s stable field — the media URL — with a deterministic hash (ch. 3). State the payoff immediately: **two devices reach the same id with no coordination**, which is what makes the rest possible.

Step 3 — Layers, with one rule each



Say what each boundary *forbids*:

- **The store** knows nothing about episodes as a domain — it holds rows.
- **The repository** is the only thing that merges, and every read path uses the same merge function (ch. 5).
- **The engine** is the only thing that owns an AVPlayer, an audio session and the Now Playing centre — and the only place every user input passes through, which is why it is the only thing that can report *provenance* (ch. 6).
- **The recorder** is the only writer of listening progress, and it lives on the app, not on a screen (ch. 6).
- **View models** own presentation and nothing else.

Step 4 — Split the schema before you are asked

Draw two stores. Say: “*the feed cache is reproducible and the download path is machine-specific, so neither syncs; favourites, positions and bookmarks exist nowhere else, so those do.*” Then name the constraint that forces the foreign key: relationships cannot cross configurations (ch. 4).

Step 5 — Conflict resolution, with the field taxonomy

The line that wins this round:

“It depends what the field *means*. A position is a measurement of a moment, so last-write-wins on a timestamp I write myself — not on arrival order, which is not event order. A flag is a decision, so decisions union, because a union converges regardless of delivery order. The union’s price is that clearing cannot propagate, and the fix for that is to store the decision’s timestamp too.”

Step 6 — The second surface, unprompted

Introduce CarPlay yourself, because it is the part that exposes whether your layering was real:

“CarPlay can launch the app with no window at all. So anything that has to happen whenever audio plays cannot live on a screen — in my case that was progress recording, and the bug was that listening in the car was never written down. And every rule about *where* an episode resumes has to live below the UI, or the car gets a different answer than the phone.”

Step 7 — Failure modes, named

- Feed unreachable → the store is the cache; the app opens with content.
- Store unopenable → tiered fallback, synced → local → memory (ch. 4).
- Schema refused by CloudKit → the app works and never syncs; **say that this needs to be visible in the UI and that in your implementation it is not** (ch. 15).
- Item fails mid-playback → rebuild at the recorded second, not from zero (ch. 7).
- Network lost mid-download → the file reference is only written on success, so a partial download never claims to be playable.

Step 8 — What you would do differently at scale

Invite this one; it shows you know the design’s edges.

- **More surfaces** (watchOS, tvOS): the engine/recorder/repository split already supports them; what does not is `PlayerViewModel`, which has grown large and mixes presentation with the account-following rules.
- **More users, real telemetry**: the analytics design (ch. 12) is built for one question. A product with many questions needs an event schema and a consent flow, not four enum cases.
- **A real backend**: the LWW/union machinery exists because there is no authority. With a server, positions become an authoritative resource with a version, and most of chapter 5 disappears. Say that — recognising when your own clever solution is a workaround for a missing component is a senior signal.

Things that will lose you the round

- Starting with MVVM/VIPER/TCA. Nobody asked about your folder structure.
- Drawing a “sync service” box with no conflict rule inside it.
- Saying “I’d use Core Data” with no answer for two devices writing the same row offline.
- Claiming everything is testable. The right answer is *what* is testable, *where* the boundary is, and *why* that boundary is where it is.

18. Behavioural stories, from real commits

Four stories, in STAR shape. Each one is true, each one has a commit behind it, and together they cover most of what behavioural rounds probe: debugging under uncertainty, disagreement, intellectual honesty, and judgement about scope.

Story 1 – “A bug that was hard to find”

Situation. Listening in the car was never recorded. Resume at 57:45, drive for fifteen minutes, come back – 57:45. Every time, only in the car.

Task. No crash, no error, no log. Audio itself was perfect. And no way to attach a debugger to a car.

Action. Worked back from the one fact available: the audio was right, so the engine was fine; the *record* was wrong, so whoever writes it was not running. Progress recording lived in the player view model, which a SwiftUI view creates – and CarPlay launches the app with a `CPTemplateApplicationScene` and nothing else. The object had never been constructed. Moved the responsibility to a `ListeningRecorder` started from `didFinishLaunching`, subscribed to the engine rather than to any view.

Result. Listening in the car is recorded. The regression test is the bug report with the user’s own numbers in it, and it deliberately constructs no view model, because the absence is the assertion.

The line to land on: *“A responsibility has to live on an object whose lifetime is at least as long as the events it is responsible for.”*

Story 2 – “Difficult code review feedback”

Situation. Three rounds of automated review on the refactor PR. Five real defects, including a crash and a sync-correctness bug – and one recommendation I did not accept: importing the previous Realm database so users would not lose favourites and bookmarks.

Task. Losing user data is not a decision to take lightly, and “the reviewer is wrong” is not a reason.

Action. Wrote the import. The old rows were keyed by identifiers with no meaning in the new schema, so every episode had to be matched by its media URL – and a mismatch writes a listening position onto the wrong episode. Silently corrupt data is worse than an empty start. Threw the import away, and wrote the reasoning into the source file rather than the PR thread, with the cost named and an obligation attached: it goes in the release notes.

Result. Accepted. And the reasoning is still in `AppDatabase.swift`, where the next person to notice the missing import will find it without archaeology.

The line: *“A rejected review finding should leave a written trace in the code, not in the pull request. The PR is read once; the file is read forever.”*

Story 3 – “A time you were wrong”

Situation. Devices were not seeing each other’s listening positions. I diagnosed a stale `ModelContext` – a context kept for the life of the app answering with rows it read before an import changed them – added a re-read before every decision, wrote a test, wrote an explanatory comment, shipped it. It appeared to work.

Task. During review, someone poked at the test. I decided to check it properly rather than defend it.

Action. Deleted the re-read and ran the test. **It passed.** The repository keeps no rows between reads – it fetches every time and hands out copies – so there was nothing stale to refresh. My fix had never done anything. The real cause had been upstream: the other device’s row had not arrived at all, and it started arriving for unrelated reasons.

Result. Rewrote the comment to say what is true, rewrote the test to assert what it actually proves, and **kept the code** – for a smaller, honest reason: it costs nothing, and it becomes necessary the day anything starts caching rows. The commit is called *“Stop claiming the re-read was the fix.”*

The line: *“If a test passes with the fix removed, the test does not test the fix. It takes five seconds to check and almost nobody does it.”*

Story 4 — “Knowing when a finding is a symptom”

Situation. Round two found one missing case: the flag that stops another device's position being recorded as your own listening was cleared by exactly one thing — a drag of our own scrubber.

Task. The obvious fix is to add the missing case. The skip buttons.

Action. Listed everything that can move the playhead: scrubber, skip buttons, lock screen `changePlaybackPositionCommand`, CarPlay. Three of the four never touch our UI at all — a press on the lock screen passes through no screen of ours. So the flag could never be maintained correctly at the UI layer, for any number of added cases. Moved the responsibility to the engine, which is the one object every input passes through, and had it publish every move a person makes. The recorder subscribes there.

Result. Four cases closed, including two nobody had reported. The fix is smaller than the one that was asked for.

The line: *“The finding was real but narrow. Treating it as a question about where the responsibility belonged, rather than as a missing case to add, is what closed the ones nobody had found.”*

Two more, in one line each

“A time you cut scope.” watchOS and tvOS were scoped and deferred to the next version rather than delaying 3.1. The engine/recorder/repository split already supports another surface; `PlayerViewModel` does not, and shipping a half-adapted view model to a watch would have cost more than waiting.

“A time you deliberately made the UI slightly wrong.” `listeningProgress` clamps to a minimum of 2 %, so a minute into a three-hour episode draws as a visible sliver rather than an invisible line. It is inaccurate by about 1.5 % and it is the only version that communicates anything. Meanwhile the same property returns `nil` for a finished episode on the phone — a full bar says nothing in a list — and CarPlay *deliberately overrides that* to draw a full bar, because from a car seat a full bar says “heard” faster than any mark could. **The same data, presented oppositely, because the reading conditions are opposite.**

Part VI – Modern Swift and SwiftUI lab

This part supplies the SwiftUI and concurrency depth needed alongside the repository walkthrough. **Current approach** means code present in this app. **Alternative** means a proposed design, not a claim that it has shipped. Complete examples below stand alone; excerpts with ... are deliberately incomplete. The deployment target is iOS 17.5, so Observation is available, but this app currently uses Combine's `ObservableObject` and `Published`.

19. SwiftUI: identity, ownership and updates

Q. How should an experienced UIKit developer think about a View struct?

A. It describes the UI for some inputs. It is not the persistent screen object that a `UIViewController` is. SwiftUI can recreate the description while retaining state in storage associated with the view's identity. Re-evaluating body is neither destroying every underlying UIKit view nor necessarily repainting every pixel. Dependency changes invalidate relevant view computations; reconciliation then decides what needs updating.

Project example. `RootView` describes a sidebar or tab layout depending on size class. Its `@StateObject` `playerViewModel` survives repeated body evaluations at the same root identity. Starting audio, fetching the feed, or writing `SwiftData` from body would make those effects repeat for reasons unrelated to user intent. Keep rendering cheap and side-effect-free.

Follow-up: Is a View initializer called only once? No. Initializers may run frequently. Do not interpret initializer/deinitializer logging of a helper as proof of a screen's visible lifetime. UIKit's view-controller lifecycle and SwiftUI identity are different abstractions.

Q. What are structural and explicit identity?

A. Structural identity comes from type and position in the hierarchy. Explicit identity is supplied by an ID, including `ForEach`'s element IDs or `.id(...)`. Identity determines which state storage SwiftUI reuses.

Project example. `RootView` gives `BookmarkToastView` `.id(bookmark.id)`. A new capture intentionally gets fresh local state. By contrast, episode rows must use the stable `Podcast.id`; a freshly generated UUID on each fetch would turn updates into replacements, losing row state and disrupting animations. `Podcast` equality includes mutable data, so equality is not a substitute for stable identity. In UIKit terms, think stable diffable-data-source identifiers, not array indexes or equality of the entire view model.

Follow-up: Is `.id(UUID())` a harmless refresh trick? No. It discards the subtree's state on every reevaluation. Use explicit identity only when that reset represents the domain, such as choosing a different episode editor.

Q. What owns state in the current project?

Mechanism	Ownership and correct use here
<code>@State</code>	View-lifetime value storage: scrubber draft value and drag offset.
<code>@Binding</code>	A read/write connection to another owner's storage; not storage itself.
<code>@StateObject</code>	Owns the lifetime of a Combine observable model for a view identity.
<code>@ObservedObject</code>	Observes an externally owned model, such as <code>AppSettings.shared</code> .
<code>@EnvironmentObject</code>	Retrieves an ancestor-injected Combine model such as the player.

Mechanism	Ownership and correct use here
@Environment	Retrieves values or dependencies, such as size class and scene phase.

Project example, PodcastEpisodesView.swift:

```
@StateObject private var viewModel: PodcastEpisodesViewModel

init(show: Show) {
    _viewModel = StateObject(
        wrappedValue: PodcastEpisodesViewModel(show: show)
    )
}
```

The declaration says this view owns the model's lifetime. The initializer supplies its initial value through `StateObject`'s deferred construction. Recreating the same view description does not replace the installed model. However, changing `show` while preserving the *same identity* does not reinitialize that model either. A destination must have appropriate `show` identity, or explicitly update its model when its input changes.

Trap. `@ObservedObject var model = Model()` constructs a model whenever that view value is constructed. The wrapper observes; it does not provide stable ownership. It is not specifically `body` that calls the initializer.

Q. Why does the scrubber keep local state instead of binding directly to the engine?

A. `ScrubberView` in `FullscreenPlayerView.swift` owns a draft slider value and an `isScrubbing` flag. During a drag, the user's draft is authoritative; periodic playback reports must not pull the thumb back. On release it sends an intent to seek; outside editing it can mirror engine progress again.

This is one source of truth per responsibility: the engine owns actual playback, the control owns an in-progress gesture. A binding to engine time would mix a user command with a measurement. The UIKit comparison is an editable text field that does not replace its text on every server refresh while the user is typing.

Follow-up: Could a binding setter start an expensive request? It can, but controls may call it repeatedly. For a seek slider, commit on edit completion or explicitly throttle. Keep the measurement path distinct from the command.

Q. How would you migrate to Observation?

A. Current approach: `PlayerViewModel` and the list models use `ObservableObject`; changes announced through `objectWillChange` can invalidate subscribers even when they do not display that particular property.

Alternative, complete standalone illustration:

```
import SwiftUI
import Observation

@MainActor @Observable
final class EpisodeFilter {
    var showsPlayed = false
}

@MainActor
struct FilterHost: View {
    @State private var filter = EpisodeFilter()

    var body: some View {
        FilterControl(filter: filter)
    }
}

@MainActor
struct FilterControl: View {
    @Bindable var filter: EpisodeFilter

    var body: some View {
        Toggle("Prikaži preslušane", isOn: $filter.showsPlayed)
    }
}
```

@Observable instruments property access and mutation. @State preserves the owned instance for this view identity. @Bindable produces the writable projection needed by Toggle; it does not own the instance. A child that only reads the model may use an ordinary stored reference. Observation tracks properties read while evaluating the view, so unrelated property changes need not invalidate that view.

Trade-off. The real AppSettings also persists preferences; replacing it with this illustrative model would lose that behavior. Migrate ownership and persistence deliberately. Observation does not make a model thread-safe, and it does not automatically replace Combine pipelines used by the recorder. @MainActor answers isolation; @Observable answers observation.

Follow-up: Must a View never own a service with @State? That is too broad. An owned @Observable reference can correctly live in @State. A service whose lifetime must exceed the screen, such as the playback engine, belongs in an app-level owner. An ordinary non-observable class in @State retains its reference, but mutating its internal properties does not by itself notify SwiftUI. Decide from lifetime and observation requirements, not from the name “service”.

Q. Is the environment dependency injection or a service locator?

A. It is a propagation mechanism with implicit lookup at consumption. The app injects the player at the root so all descendant controls share it. That avoids threading a parameter through every intermediate view, but hides requirements from initializer signatures. A missing @EnvironmentObject crashes at runtime, including in previews. Test and preview roots must inject it. For domain services, explicit initializer injection makes dependencies and test doubles easier to see; use environment for genuinely shared UI scope. Typed Observation injection uses `.environment(model)` with `@Environment(Model.self)`, not `@EnvironmentObject`.

Q. How should navigation and presentation be modeled?

A. Current approach: show selection leads to PodcastEpisodesView; the expanded player is an overlay driven by `PlayerViewModel.isExpanded`, so audio can continue across tab changes and collapse. This is presentation state, not a second playback engine.

Alternative: if the app needs deep links and restoration of multi-step routes, use a typed `NavigationStack` path whose values are stable domain IDs. Resolve IDs at the destination and handle an episode missing from the local cache. For a selected editor, `.sheet(item:)` makes the selected item and its presentation one optional state value, avoiding contradictory booleans.

Trade-off. A navigation coordinator is useful when route policy is shared; adding one just to wrap a two-screen stack adds indirection. Do not keep full `SwiftData` model objects as navigation state across unrelated contexts.

Q. How do `.task`, `onAppear`, and `scenePhase` differ?

A. `onAppear` is a visibility callback and can run more than once. In the current episode view it calls `fetchData()`, whose model must prevent duplicate loads. `.task` starts async work associated with the view's lifetime and requests cancellation when that lifetime ends; `.task(id:)` also restarts when its ID changes. Cancellation is cooperative, not a guarantee that every operation has stopped before the next one begins. `scenePhase` describes a scene's activity, not whether a particular view is on screen.

Project implication. Re-fetching a list is appropriate view work. Audio playback and recording a CarPlay listen cannot depend on a view task that may not exist. `ListeningRecorder` therefore starts at application launch.

Follow-up: Does `Task {}` inside `onAppear` inherit view cancellation? No. That creates an unstructured task. Keep and cancel its handle if that is the intended ownership, or use the view's `.task` modifier for view-scoped work.

Q. What would you profile before optimizing this UI?

A. Reproduce scrolling with playback active, a long episode list, and incoming store updates. Use Instruments to distinguish main-thread CPU, layout, image decoding, database fetches, and excessive view invalidations. A frequent `body` evaluation is evidence to investigate, not automatically a defect.

Project example. The player publishes progress frequently, while rows mostly need current-episode identity and playback state. Splitting those observations can reduce unnecessary work. Keep row IDs stable, avoid sorting or fetching all episodes inside `body`, and decode/resize artwork once rather than per frame. Measure release builds on a device as well as the simulator. A 60 Hz frame is about 16.7 ms; 120 Hz leaves roughly half that time.

Follow-up: Does `withAnimation` move work off the main actor? No. It supplies an animation transaction to state changes. It cannot make expensive synchronous work cheap. Respect `Reduce Motion`, avoid animating every periodic time update, and use transitions only when insertion/removal and identity are intentional.

Q. How would you migrate a UIKit screen incrementally?

A. Embed a SwiftUI feature in `UIHostingController`, keeping its dependencies owned by the surrounding composition root. Expose explicit actions back to the UIKit coordinator. In the other direction use `UIViewRepresentable` or `UIViewControllerRepresentable`, with `make` for creation, `update` for applying current inputs, a coordinator for delegates, and `teardown` for observers. The coordinator must not keep stale copies of changing inputs or create a retain cycle through callbacks. The repository already retains UIKit lifecycle integration through `AppDelegate`; a rewrite of the whole app is unnecessary.

20. Concurrency: execution, cancellation and isolation

Q. What actually happens at `await`?

A. It marks a possible suspension point. If the operation suspends, the task preserves its continuation and releases its executing thread to other work. It resumes when the awaited operation completes, on the executor required by isolation. It does not block a thread like a semaphore wait. An `await` may complete without suspending; it is not a mandatory context switch.

Project example. An async replacement for `PodcastService` could suspend while `URLSession` waits for network data. Decoding a large response remains CPU work. Calling an async method does not automatically move decoding off the main actor, and wrapping synchronous decoding in `Task {}` from a main-actor context does not solve that problem.

Version trap. This project uses Swift 5 language mode. Swift 6.2's `NonisolatedNonsendingByDefault` feature changes unannotated nonisolated async execution to remain on the caller's actor; earlier semantics use the generic executor. Check language mode and enabled features. With modern semantics, `@concurrent` explicitly requests concurrent execution for CPU work. State the configuration before claiming that a nonisolated async function leaves main.

Q. How do structured and unstructured tasks differ?

Form	Lifetime and suitable project use
<code>async let</code>	Fixed child operations whose results are awaited before leaving scope.

Form	Lifetime and suitable project use
Task group	Dynamic children, bounded by their enclosing scope.
Task {}	Unstructured task; caller owns lifecycle explicitly. Inherits surrounding isolation when applicable.
Task.detached	Unstructured; does not inherit actor isolation or task-local values. Requires an explicit ownership and cancellation plan.

A. Child tasks cannot outlive their structured scope. Scope exit waits for them, including after cancellation. An unstructured task is not automatically cancelled because its creator returns or is cancelled. Detached does not mean “dedicated thread” or “guaranteed faster”. For this app, a screen load is view-owned work; a download intentionally continuing after navigation needs an application-level owner and a user-visible cancel operation.

Follow-up: Does a throwing group instantly stop siblings? No. When an error propagates out of the group body, remaining children are cancelled and awaited. Children must cooperate. You must consume throwing results; an error is not a magical immediate broadcast just because a child throws. A race that returns the first answer still waits for cancelled children to finish before leaving its structured scope.

Q. What is cooperative cancellation?

A. `cancel()` records a cancellation request. Code checks `Task.isCancelled`, calls `Task.checkCancellation()`, or awaits an API that responds to cancellation. A CPU loop must check periodically. Cleanup still runs. Cancellation is distinct from a user-visible network failure.

Project alternative. A feed request should cancel when its screen-owned task disappears. A response for an old show must never overwrite a newer selection. Use cancellation *and* a request identity check: callback adapters may not stop promptly, and an old result can race a new request.

Follow-up: Is weak self sufficient? No. `guard let self` before a long `await` holds the object strongly through that suspension. A model holding a task whose closure holds the model can keep itself alive indefinitely, especially when consuming an endless stream. Capture a service independently, use an explicit owner to cancel, and avoid relying only on `deinit` to break a cycle that prevents `deinit` from running.

Q. How would an async network boundary look here?

A. Current approach: `NetworkManaging` uses callbacks and has inconsistent callback-queue contracts across overloads. Its API does not expose the task needed to cancel a request. The review fixes session construction so the configured timeouts actually apply; it does not silently migrate callers to `async/await`.

Alternative, standalone example:

```
import Foundation

enum FeedFailure: Error {
    case invalidResponse
    case http(Int)
}

struct FeedClient: Sendable {
    func data(for url: URL) async throws -> Data {
        var request = URLRequest(url: url)
        request.timeoutInterval = 15
        let (data, response) = try await URLSession.shared.data(for: request)
        try Task.checkCancellation()
        guard let http = response as? HTTPURLResponse else {
            throw FeedFailure.invalidResponse
        }
        guard (200..<300).contains(http.statusCode) else {
            throw FeedFailure.http(http.statusCode)
        }
        return data
    }
}
```

The request carries its timeout before submission. The suspension waits for transport. The cancellation check prevents processing a just-cancelled result. HTTP validation is explicit because a completed transport can still return a 500 response. Returning `Data` separates transport from decoding and state mutation. The real app would inject a session for protocol-based tests rather than hard-code `.shared`.

Trade-off. `data(for:)` buffers the response. For large media use download APIs or streaming bytes, not whole-file `Data` in memory. Retry idempotent feed reads with bounded backoff; do not treat cancellation as a reason to retry. Keep cached episodes visible on failure and offer an actionable retry state.

Q. What does `@MainActor` guarantee?

A. It establishes isolation for state and operations, with main-executor serialization. In Swift concurrency, accessing isolated state from outside requires crossing that boundary appropriately. It is not a spell that makes an arbitrary synchronous legacy callback hop queues. Annotate the owner and adapt callback entry points deliberately; enable strict concurrency checking to find violations rather than assuming runtime thread conventions suffice.

Project example. Playback state, UI models, and the current synchronous store layer are intended for main-thread access. AVFoundation completion callbacks need an explicit hop before changing `@Published` properties. The seek review adds that hop and invalidates old completions on teardown.

Follow-up: Should decoding run on `MainActor`? Small decoding may be fine; measure. Large decoding and full-library transformations can block input even inside an async method. Move CPU work to a deliberately non-main execution boundary and transfer value snapshots back. Never move a live `ModelContext` or managed model instance into a detached task just to silence a hang.

Q. Are actors transactions across an `await`?

A. No. They serialize access to isolated state, but can process other work while a method is suspended. This reentrancy is useful for throughput and requires rechecking assumptions after suspension.

Alternative example: deduplicating feed downloads. An actor checks whether an episode is downloaded, awaits the network, and then writes the path. Another request can arrive during that `await` and also start a download. Record an in-flight task *before* suspension, or recheck and resolve duplication after suspension. Make failure remove that in-flight entry so retry is possible.

Follow-up: Does an actor have its own thread? No. An actor provides an isolation domain and executor scheduling, not a dedicated OS thread. Actors also do not solve cross-device ordering: the CloudKit merge policy is still necessary because two processes do not share an actor.

Q. What is `Sendable`, and what is it not?

A. It describes values safe to transfer across concurrency domains. Compiler-checked value types require their stored values to satisfy the relevant rules; an immutable reference or synchronized reference can also

be suitable. `@Sendable` constrains closure captures. It is not a scheduling mechanism and does not mean “run in the background”.

Project example. Prefer passing an episode ID or immutable DTO into a background worker and returning a value result. Podcast is a useful candidate for a checked conformance after auditing its stored properties. SwiftData entities and contexts are not interchangeable with those snapshots. `@unchecked Sendable` is a manual proof obligation, not a warning suppression strategy. Audit all mutation paths and callbacks before using it.

Q. How do you bridge a one-shot callback with a continuation?

A. A checked throwing continuation suspends the task and must be resumed exactly once, with a result or error. It does not add cancellation to the callback API. A correct adapter needs the underlying cancel handle, a cancellation handler, and synchronization for completion-versus-cancel races. Cancellation can arrive before that handle is installed. Both paths must share a once-only completion state.

Project trap. `NetworkManaging` currently returns `Void`, so a naïve `withCheckedThrowingContinuation` wrapper cannot cancel the underlying request. Expose the task or use native async URLSession APIs. The seek callback is the sharper trap: a superseded seek does hand control back, but one whose player has been torn down is dropped on purpose (ch. 7), so a direct `withCheckedContinuation` wrapper around it would leak a suspended task on every `stop()`. Wrapping it needs the teardown path to resume the continuation with cancellation.

Follow-up: When is an unsafe continuation appropriate? Only after a measured need and a proven exactly-once contract. Checked continuations help diagnose misuse; neither kind repairs missing callbacks or races automatically.

Q. When would `AsyncSequence` or `AsyncStream` help?

A. An async sequence supplies multiple values over time through `for await`. For this app, download progress or live metadata is a better fit than a one-shot continuation. `AsyncThrowingStream` can additionally end with an error. A stream does not automatically make its producer structured or cancel external observers when the consumer leaves.

Alternative design. Adapt a download delegate into a progress stream with `.bufferingNewest(1)` because the UI needs the latest percentage, not thousands of stale samples. Use `onTermination` to release the delegate/observation and cancel work if this stream owns it. Serialize cleanup with delegate callbacks; termination may arrive on a different executor. Finish the stream on completion and handle its terminal result separately from progress.

Trade-off. Dropping intermediate *events* can lose information, so do not reuse that buffering policy for bookmark commands. Multiple consumers of one `AsyncStream` are not automatically a multicast state publisher. Keep `Combine` where its current-value and multicast semantics are part of the contract, or explicitly design those semantics in the replacement.

Q. How would you test async behavior without arbitrary sleeps?

A. Inject a transport, clock, and storage boundary. Suspend a fake request until the test releases it; change the selection; complete the obsolete request; assert it cannot overwrite the new state. Cancel during a request and assert cleanup and the absence of a displayed cancellation error. For time-based metadata expiry, advance an injected clock instead of waiting ten minutes. Use async XCTest fulfillment when the producer needs the main actor; blocking that actor waiting for work scheduled on it can deadlock.

Project connection. Existing tests inject engine and store protocols and use expectations for real `AVPlayer` behavior. Keep a small framework integration suite because a fake engine cannot prove seek semantics. Add a regression test that stops or replaces the player before an outstanding completion arrives. Use Thread Sanitizer and Instruments for additional evidence; passing a unit test is not proof that every callback is race-free.

21. A three-to-five-minute project walkthrough

Q. Tell me about this project and the decisions you would defend.

A. AlarmDM is a radio and podcast client with live audio, downloaded listening, favorites, and bookmarks. The same product supports iPhone, iPad, Mac, and CarPlay. The engineering problem is making one listening session behave consistently across UI entry points and eventually across devices.

The playback engine owns the active `AVPlayer`, audio-session interaction and Now Playing integration. The

phone UI, CarPlay and remote commands all issue intents through it. The UI does not own the lifetime of playback. A separate ListeningRecorder observes the engine and persists progress even when CarPlay launches the application without a phone window. That boundary came from a real failure: audio worked in the car while nobody saved where it got to.

SwiftUI uses view-owned observable presentation models and a shared player facade. Lists operate on value snapshots, with stable IDs rather than generated identities on every refresh. The scrubber temporarily owns a gesture draft so periodic playback observations cannot fight the user's drag. Wide layouts use a sidebar and persistent player area; phone layouts use tabs and an expanded player overlay. UI adaptation does not duplicate playback policy.

SwiftData has two store configurations. Reproducible feed data and device-local file paths stay local; listening state and bookmarks sync through CloudKit. The repository folds duplicate state rows consistently for both list and single-episode reads. The current position selection uses timestamps; boolean flags are merged with OR. That is a deliberate but incomplete conflict policy: removing a favorite needs a more expressive versioned decision, and equal timestamps need a deterministic tie-breaker before claiming full convergence. CloudKit transport does not remove those domain decisions.

Networking is still callback-based. The near-term correctness fix is creating a URLSession with its configured timeouts, rather than mutating the copy returned by an existing session. The next architectural step is explicit isolation and cancellation-aware request APIs, with DTOs crossing execution boundaries. I would keep framework ownership and domain behavior stable while migrating, rather than rewrite all layers at once.

Testing follows those boundaries: fake engine/store tests cover playback and sync decisions; in-memory and two-container tests cover repository behavior; a small real-player test checks EOF and replay assumptions. These do not prove CloudKit delivery, device audio interruption behavior, or every UI layout, so release validation still includes devices and platform-specific interaction. For performance I would measure main-thread stalls and repeated observation updates while scrolling with audio active. For memory I would trace observer tokens, subscriptions and long-lived task ownership. My next improvements are compiler-enforced isolation, visible storage fallback, cancellation-aware networking, and a conflict policy for reversible user decisions.

22. Interview cheat sheet

Topic	Answer to remember
View value	A description; identity-backed storage outlives many descriptions.
Identity	Stable domain ID; changing identity intentionally resets local state.
State / Binding	Owned storage versus access to another owner's storage.
StateObject	Stable owner of a Combine observable model for one view identity.
Observation	Tracks property reads; @State owns, @Bindable exposes bindings.
Environment	Shared UI scope; explicit injection remains useful for services/tests.
Lifecycle	onAppear repeats; .task requests cancellation; playback outlives views.
Await	Possible suspension, not a blocking wait or automatic background thread.
Structured work	Children are scoped and awaited; cancellation is cooperative.
Task / detached	Unstructured; define ownership, retention and cancellation explicitly.
MainActor	Isolation; CPU work can still block UI even inside an async method.
Actors	Serialized state access; invariants spanning await must be rechecked.
Sendable	Transfer safety, not scheduling; audit captured and stored references.

Topic	Answer to remember
Continuation	Exactly once; cancellation and callback races need a real design.
AsyncStream	Lifetime, cleanup and buffering are part of the API contract.
Persistence	Confine contexts; move snapshots/IDs across isolation boundaries.
Networking	Validate HTTP, distinguish cancellation, bound retries, preserve cache.
Memory	Trace owner to task/subscription to closure to owner; weak is not enough.
Performance	Profile release builds; body counts alone are not frame-cost evidence.
Testing	Control time/completions; supplement fakes with real framework tests.
Architecture	Explain ownership, lifetime, failure policy and a concrete trade-off.

Primary references

Use these primary references to check platform claims against the SDK being used; the codebase's Swift language mode is not the same as the compiler version.

- SwiftUI model data: <https://developer.apple.com/documentation/swiftui/managing-model-data-in-your-app>
- Swift language concurrency: <https://docs.swift.org/swift-book/documentation/the-swift-programming-language/concurrency/>
- Swift SE-0461, async isolation semantics: <https://github.com/swiftlang/swift-evolution/blob/main/proposals/0461-async-function-isolation.md>
- AVPlayer seek completion: [https://developer.apple.com/documentation/avfoundation/avplayer/seek\(to:completionhandler:\):](https://developer.apple.com/documentation/avfoundation/avplayer/seek(to:completionhandler:):)
- App Tracking Transparency scope: <https://developer.apple.com/app-store/user-privacy-and-data-use/>

The modern examples are alternatives, not excerpts from shipped production code. Existing historical excerpts document the behavior at the cited commit. The review corrects claims where those comments or narratives overstated what the implementation or a test actually proves.

Appendix A — File map

Where to look when you want the real thing rather than the excerpt. Paths are relative to AlarmDM/AlarmDM/.

File	What it is good for
Player/PlaybackEngine.swift	The whole of chapter 7. Also <code>PlaybackSource</code> , <code>LiveTrack</code> , remote commands, Now Playing, audio session, interruptions. ~860 lines and the densest file in the project.
Player/ListeningRecorder.swift	Chapter 6, end to end. Short and worth reading whole.
Store/AppDatabase.swift	Chapter 4. Two configurations, the tiered fallback, remote-change observation, the Realm decision, the DEBUG erase.
Store/PodcastRepository.swift	Chapter 5's writes and reads, <code>resumePosition</code> , <code>lastListened</code> , <code>merged</code> , <code>commit</code> .
Store/EpisodeStateEntity.swift	The merge rule itself — effective and <code>isNewer</code> . Six lines that matter.
Model/Podcast.swift	Chapter 9. All the listening rules, plus <code>Show</code> and its metadata.
Helpers/AppConstants.swift	<code>UUID.stable(from:)</code> — chapter 3.
Helpers/AppLanguage.swift	Chapter 11.
Helpers/Analytics.swift	Chapter 12, complete in 95 lines.
CarPlaySceneDelegate.swift	Chapter 10's CarPlay half. Templates, retry, list items, progress bars.
Store/LiveBookmarkMatcher.swift	The prettiest piece of pure logic in the project — placing a live bookmark inside an episode from <code>airedAt</code> , with no schedule involved. Untested; see ch. 15.
Views/PlayerView/PlayerViewModel.swift	Restore, follow-the-account, <code>adoptSyncedPosition</code> . Also the largest view model and the one most in need of splitting.
Views/Shared/EpisodeRowActions.swift	Runtime platform adaptation — chapter 10.
AlarmDMTests/PlayerStateTests.swift	Player state, the recorder, live track parsing, and all the doubles.
AlarmDMTests/SyncStateTests.swift	Merge rules, two-container store change, schema acceptance.
AlarmDMTests/ListeningRulesTests.swift	25 tests on the listening rules, plus 6 on which rows a list shows. No fixtures beyond struct literals.
Info.plist	The CarPlay scene declaration, <code>UIApplicationSupportsMultipleScenes</code> , background audio.

Numbers worth having ready

Swift lines	~11 000
-------------	---------

Test lines	~1 700, in 96 tests
Swift files	53
Localized keys	147, source sr-Latn
Language mode	Swift 5 (SWIFT_VERSION = 5.0), strict concurrency off
<code>#if targetEnvironment(macCatalyst)</code>	6 files of 47
Deployment target	iOS 17.5
Third-party dependencies	1 (Aptabase, optional at compile time)
Surfaces	iPhone, iPad, Mac Catalyst, CarPlay

Appendix B — Recurring themes

Five ideas turn up in three or more unrelated places. Naming a pattern you found yourself, across your own codebase, is one of the strongest things you can do in an interview.

1. Absence and zero are different values. `CMTIME` reporting `NaN` when an item fails is *no position*, not position zero (ch. 7). Backend `outroSeconds == 0` is *unmeasured*, not measured-as-none (ch. 9). A metadata callback that fired with nothing in it is not the same as one whose contents all failed to parse (ch. 8). A `PlaybackState` written before this app knew about dates is treated as `.distantPast` rather than as `now` (ch. 6).

2. One rule, read from every path. The merge function used by both the list and the single-episode read (ch. 5). `resumePosition` in the repository so the car cannot have its own (ch. 10). The bar and its label deriving their `nil` from the same predicate (ch. 9).

3. Provenance beats value. Whether a position is “a listen” cannot be answered by comparing seconds, because an adopted seek lands within a second of a genuine one. It is answered by a flag that records *where the number came from* (ch. 6).

4. A push feed that reports starts makes you responsible for the ends. Song announcements with a ten-minute lifetime (ch. 8). The same shape as a cache TTL or a presence timeout.

5. Lifetime determines responsibility. Progress recording cannot live on a screen that CarPlay never creates (ch. 6). Block observers must be held and released, which only showed up under tests (ch. 4). A periodic time observer must be removed before its player goes (ch. 7).

Appendix C — Rebuilding this document

```
cd docs/interview-guide
bash build.sh          # pandoc + xelatex → Senior_iOS_Interview_Guide.pdf
```

`guide.md` is the single source. `header.tex` carries the LaTeX layout — code-block framing, line wrapping inside listings, running headers. `swift.xml` is a Swift syntax definition written for this document, because the pandoc on this machine predates Swift support and every code block was otherwise falling back to unhighlighted verbatim. `progress.md` records what is written and what is not.

Provenance. Parts I–V were written against commit `0e5639d` and carried forward through `9af1db0` (the replay and carry-on work), `3b4735a` (the session-configuration and seek-threading review) and `e64c568` (the seek completion split). Part VI is a supplement rather than a walkthrough: its “alternative” designs are proposals, not code that shipped. Excerpts are sometimes abbreviated, and historical or proposed snippets say so where they appear.

If an excerpt and the code disagree, **the code is right** and the guide is stale.